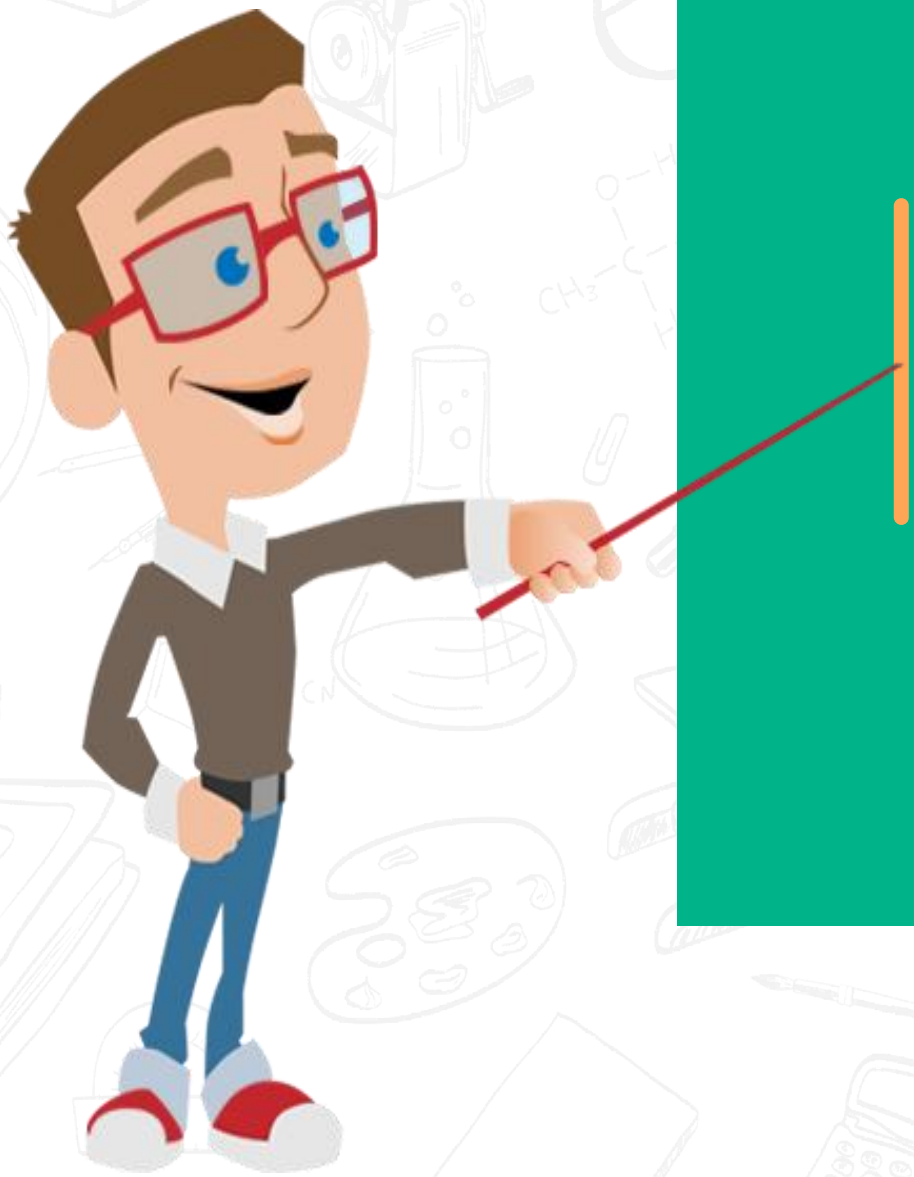


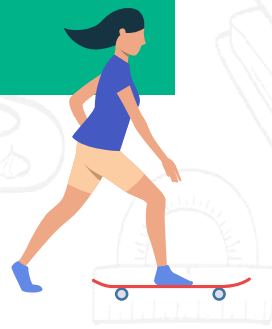
机器人操作系统（双语）

Robot Operating System (Bilingual)

WHERE THERE'S A WILL, THERE'S A WAY

授课教师：郄(qiè)龙飞

机器人工程



Overview lesson 7-1

1. CMakeLists.txt Introduction
2. ~
3. ~
4. ~

Objectives

1、掌握 CMakeLists.txt 的编译规则

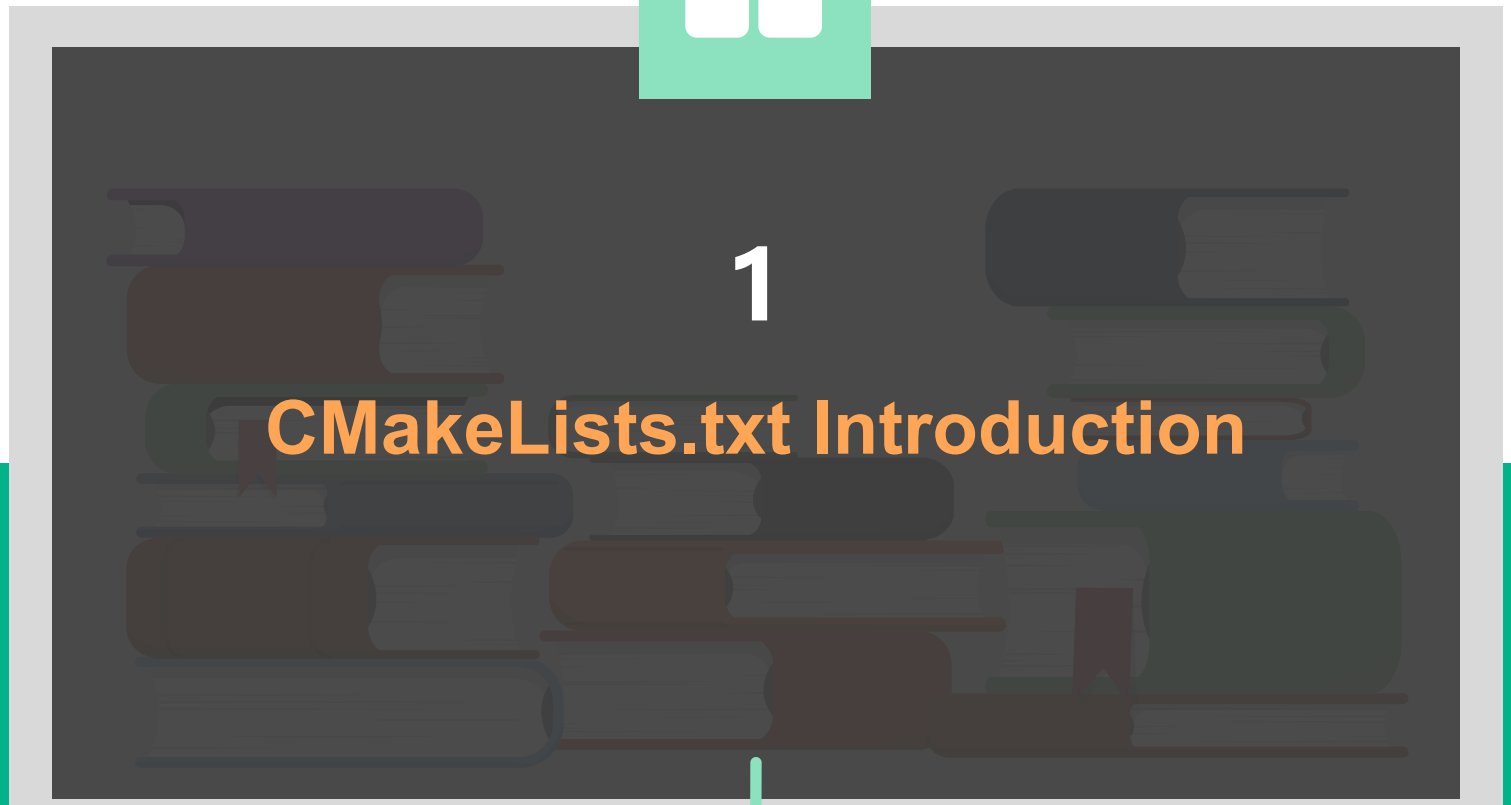
2、掌握 CMakeLists.txt 修改方法



Pre-assessment

1、是否了解 Cmake?

2、是否编写过 CMakeLists.txt?



1

CMakeLists.txt Introduction

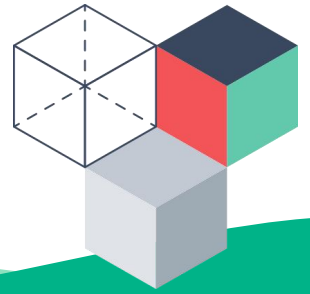
1.1 CMake

1.1.1 Overview： http://wiki.ros.org/catkin/CMakeLists.txt

- Linux 平台编译 C++ 文件，需要编译器和链接器，其中编译器是将源代码编译成目标代码，链接器是将目标代码链接到可执行文件
- 若是编译单个文件，用 g++ 即可；若是编译一个 C++ 工程，需要批处理编译工具，如 make，通过设定规则文件 makefile 调用 g++ 等编译工具进行批量编译
- makefile 的编写十分复杂，便诞生了 CMake，通过编写简单的 CMakeLists.txt 规则文件，就能自动生成 makefile 文件



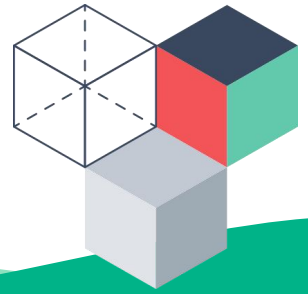
CMake
Cross-platform Make



1.1 CMake

1.1.1 Overview： http://wiki.ros.org/catkin/CMakeLists.txt

- ROS 的编译器是 CMake，ROS 在 CMake 基础上封装了 catkin 命令，用 cmake 命令创建功能包时，会自动生成 CMakeList.txt 文件，已配置了多数编译选项，且包含详细的注释，只需稍作修改便可编译自己的文件
- CMakeList.txt 规定了这个 package 要依赖哪些 package，要编译生成哪些目标，如何编译等由源码到目标文件的规则
- 工作空间两种 CMakelist.txt 文件，一个是针对所有 package（如何将所有 package 功能包生成的可执行文件组成我们想要的目标文件——宏观把握），一个是针对于某一个特定的 package（如何将某一个特定的package功能包中的源代码生成可执行目标文件——细节掌控）

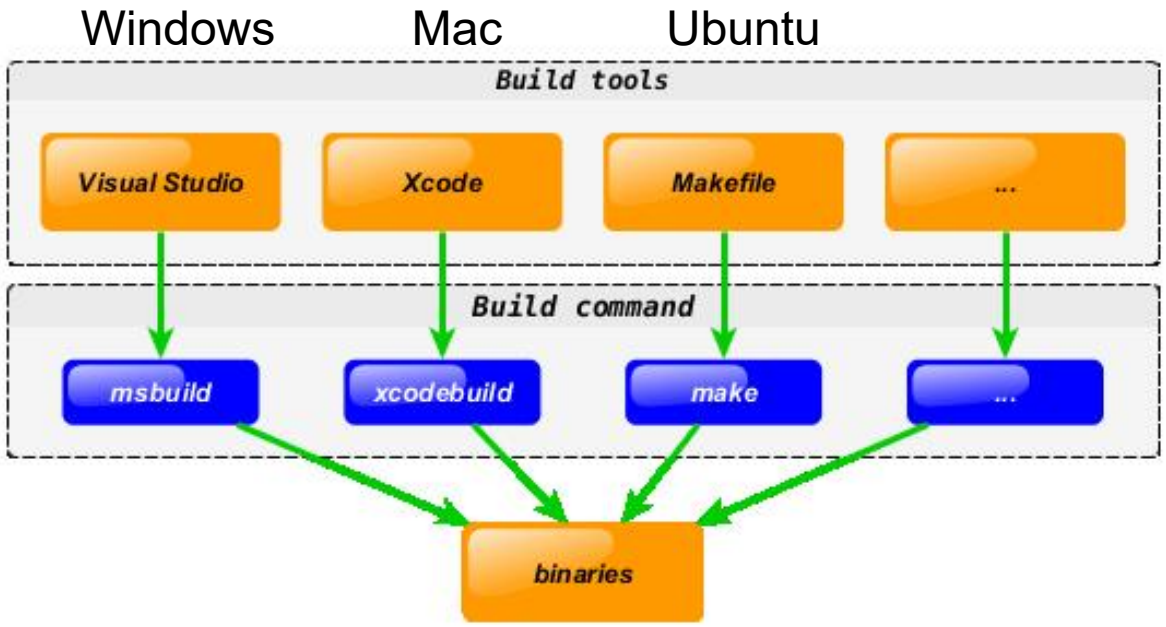


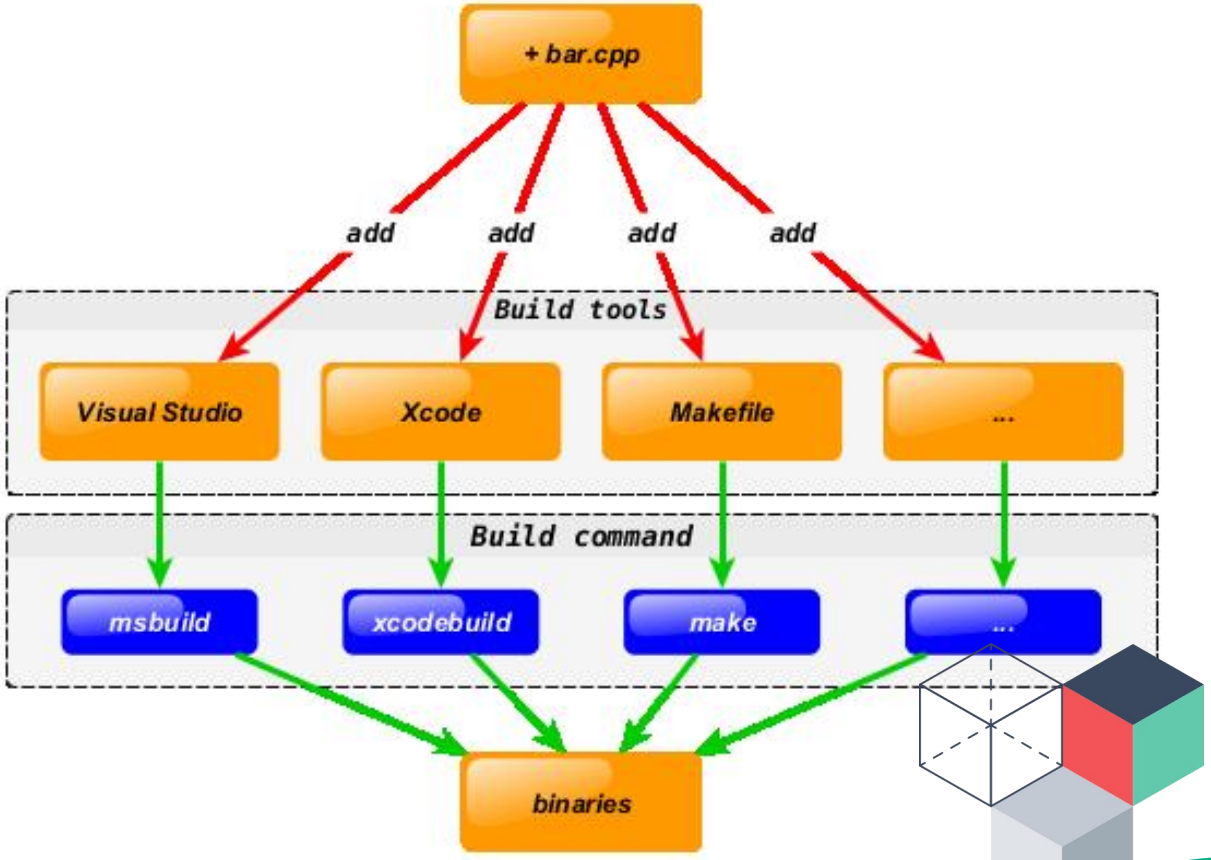
1.1 CMake

1.1.1 Overview： http://wiki.ros.org/catkin/CMakeLists.txt

- CMake是一个跨平台的C/C++编译工具

Let’s assume you have some cross-platform project with C++ code shared along different platforms/IDE.Say you use visual studio on Windows,xcode on Mac and Makefile for Linux

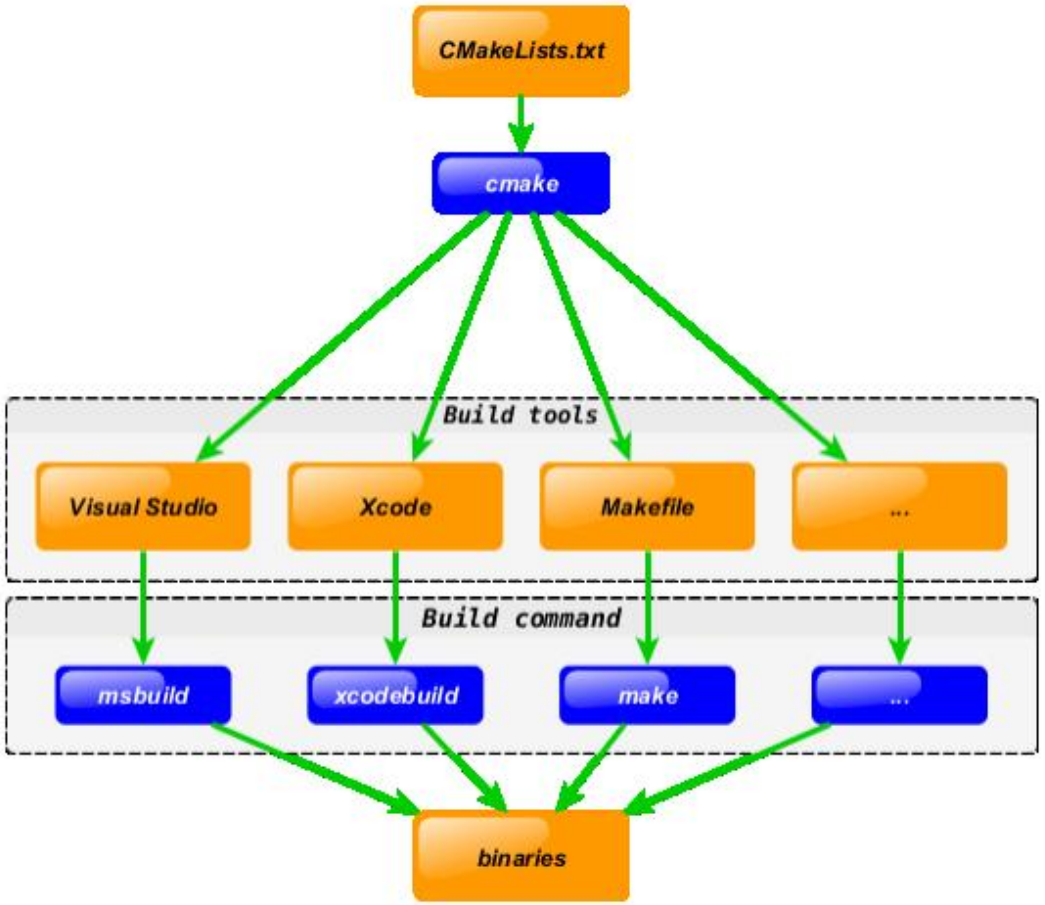


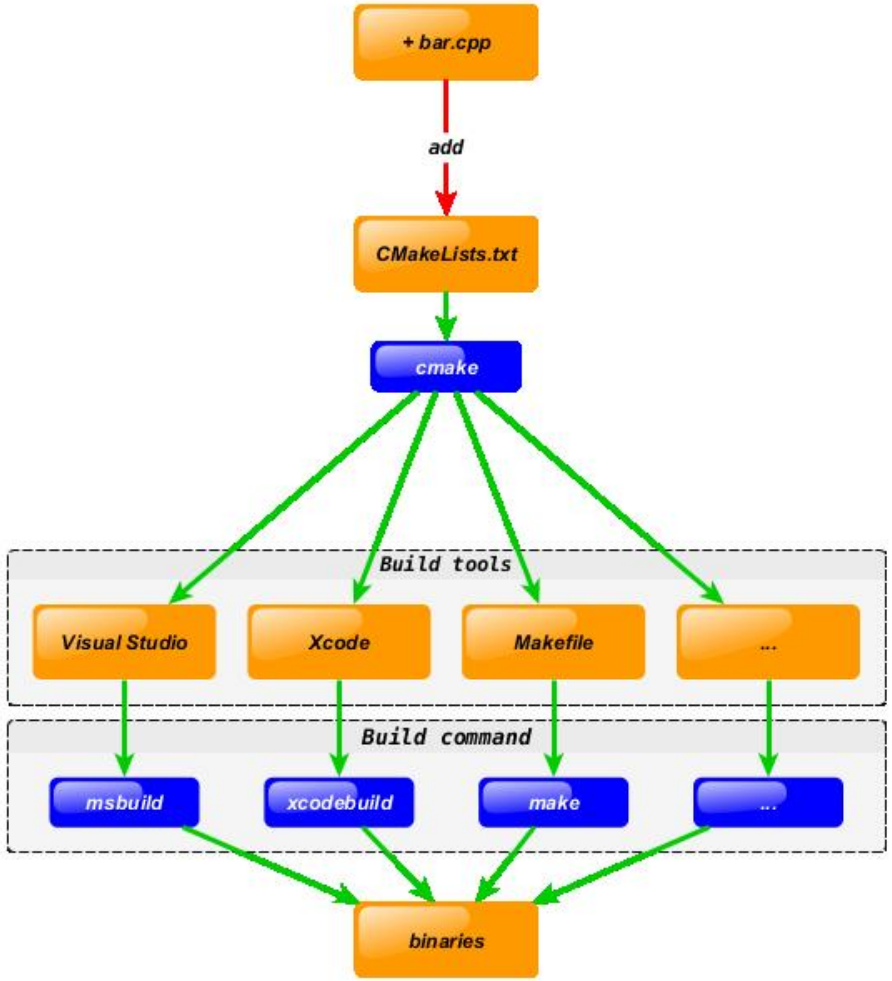


1.1 CMake

1.1.1 Overview： http://wiki.ros.org/catkin/CMakeLists.txt

- CMake可以用简单的语句来描述所有平台的安装/编译过程，自动生成对应的编辑文件

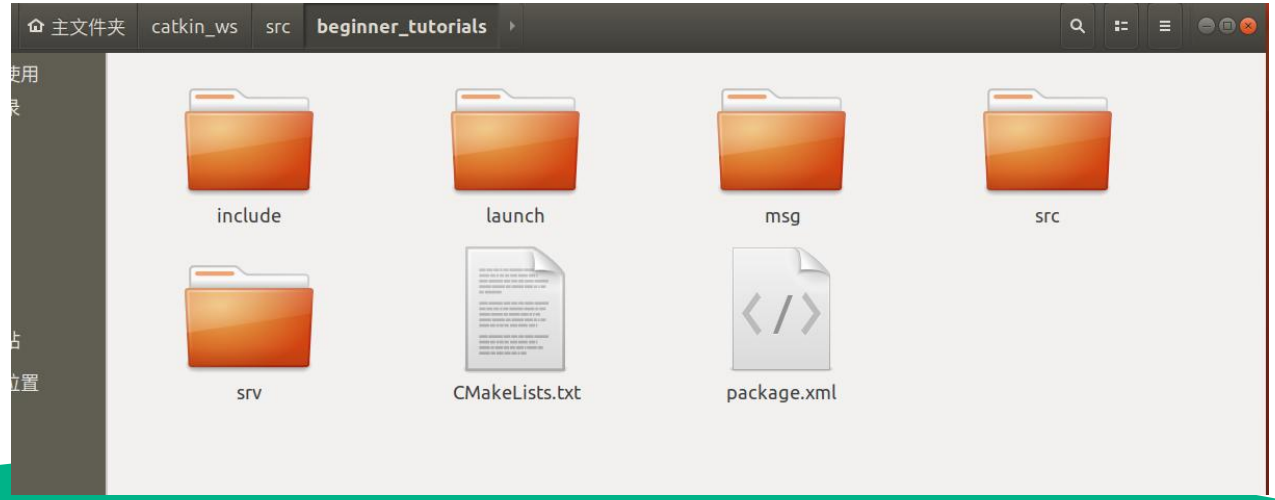




1.1 CMake

1.1.1 Overview： http://wiki.ros.org/catkin/CMakeLists.txt

- 文件 CMakeLists.txt 是用于构建软件包的 CMake 配置信息。任何符合 CMake 的包都包含一个或多个 CMakeLists.txt 文件，这些文件描述了如何构建代码以及将代码安装到何处
- ROS 使用 CMake 进行程序编译的时候，会根据 CMakeLists.txt 文件进行逐步处理，形成一个 Makefile 文件，系统再根据 Makefile 文件的设置进行程序的编译
- 在编写 CMakeLists.txt 时必须遵循特定的格式，否则软件包将无法正确构建和编译， 配置中的顺序不能改变




```
cmake_minimum_required(VERSION 3.0.2)
project(beginner_tutorials)

## Compile as C++11, supported in ROS Kinetic and newer
# add_compile_options(-std=c++11)

## Find catkin macros and libraries
## if COMPONENTS list like find_package(catkin REQUIRED COMPONENTS xyz)
## is used, also find other catkin packages
find_package(catkin REQUIRED COMPONENTS
  roscpp
  rospy
  std_msgs
  message_generation
)

## System dependencies are found with CMake's conventions
# find_package(Boost REQUIRED COMPONENTS system)


## Uncomment this if the package has a setup.py. This macro ensures
## modules and global scripts declared therein get installed
## See http://ros.org/doc/api/catkin/html/user_guide/setup_dot_py.html
# catkin_python_setup()

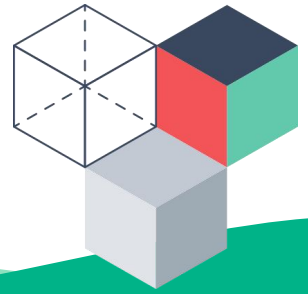
################################################
## Declare ROS messages, services and actions ##
```

1.1 CMake

1.1.1 Overview http://wiki.ros.org/catkin/CMakeLists.txt

- Cmake 语法特性：
  - 基本语法格式：指令(参数 1 参数 2...)
    - 参数使用括号括起来
    - 参数之间使用空格或分号分开
  - 指令是大小写无关的，参数和变量是大小写相关的

```
1 set(HELLO hello.cpp)
2 add_executable(hello main.cpp hello.cpp)
3 ADD_EXECUTABLE(hello main.cpp ${HELLO})
```



1.1 CMake

1.1.1 Overview： http://wiki.ros.org/catkin/CMakeLists.txt

Required CMake Version (cmake_minimum_required)

Package Name (project())

Find other CMake/Catkin packages needed for build (find_package())

Enable Python module support (catkin_python_setup())

Message/Service/Action Generators (add_message_files(), add_service_files(), add_action_files())

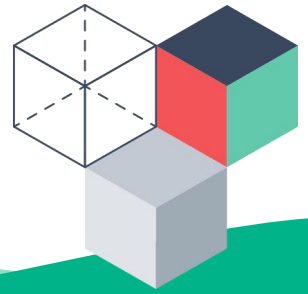
Invoke message/service/action generation (generate_messages())

Specify package build info export (catkin_package())

Libraries/Executables to build (add_library()/add_executable()/target_link_libraries())

Tests to build (catkin_add_gtest())

Install rules (install())



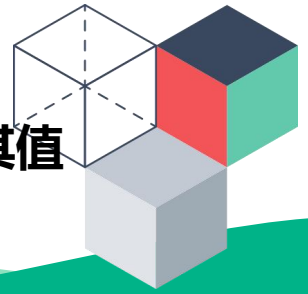
1.1.2 Required CMake Version (cmake_minimum_required)

- cmake_minimum_required - 指定CMake的最小版本要求
- 语法： cmake_minimum_required(VERSION versionNumber [FATAL_ERROR])

  下图表示 Catkin 需要 3.0.2 或更高版本

```
cmake_minimum_required(VERSION 3.0.2)
```

- 这条指令会自动创建两个变量：
  - <projectname>_BINARY_DIR(二进制文件保存路径)
  - <projectname>_SOURCE_DIR（源代码路径）
  - cmake系统也帮助我们预定义了PROJECT_BINARY_DIR和PROJECT_SOURCE_DIR其值与上述对应相等



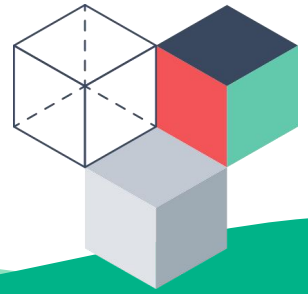
1.1.3 Package Name (project())

- project - 定义工程名称，并可指定工程支持的语言
- 语法： project(projectname [CXX] [C] [Java])

  此处功能包名称为：beginner_tutorials

```
project(beginner_tutorials)
```

- 在 CMake 中指定后，你可在其他地方通过使用变量 ${PROJECT_NAME} 来引用项目名称



1.1.4 Find other CMake/Catkin packages needed for build (find_package())

- CMake 的 find_package 函数用来指定在构建项目过程中依赖了哪些其他的系统依赖包和功能包
- 在 ROS 中，使用 catkin_make 的编译方式，至少需要 catkin 这个包，即：

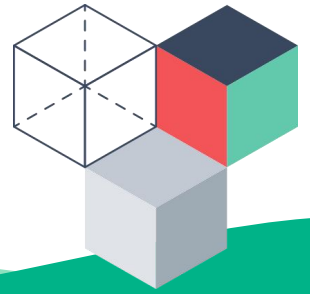
  find_package(catkin REQUIRED)

  catkin 是被搜索的项，REQUIRED表示前面的 catkin 的必须项目
- 包含其他功能包：

  find_package(roscpp REQUIRED)

  find_package(rospy REQUIRED)

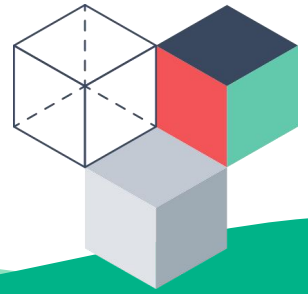
  find_package(std_msgs REQUIRED)



注意：这里添加的包用于构建功能包时使用，而不是添加运行时依赖项

1.1.4 Find other CMake/Catkin packages needed for build (find_package())

- 当一个功能包被 find_package 之后，则会创建多个 CMake 环境变量，提供有关找到功能包的信息
- 环境变量描述了功能包导出头文件的位置、源文件的位置、功能包依赖的库以及这些库的路径等
- 变量按以下格式命名：<PACKAGE NAME>_<PROPERTY>
  - <PACKAGE NAME>_FOUND - Set to true if the library is found, otherwise false
  - <PACKAGE NAME>_INCLUDE_DIRS or <NAME>_INCLUDES - The include paths exported by the package
  - <PACKAGE NAME>_LIBRARIES or <NAME>_LIBS - The libraries exported by the package
  - <PACKAGE NAME>_DEFINITIONS - Add compilation options and macro definitions



1.1.4 Find other CMake/Catkin packages needed for build (find_package())

- 那么，每个依赖的 package 都会产生几个变量，例如：

  roscpp_INCLUDE_DIRS, roscpp_LIBRARIES，rospy_INCLUDE_DIRS, so on

- 以上这种方式对变量的使用和管理都非常不便，常采用另外一种方法，即将依赖的功能包变成 catkin的组件：

```
find_package(catkin REQUIRED COMPONENTS
        roscpp
        rospy
        std_msgs
        message_generation)
```

```
find_package(catkin REQUIRED COMPONENTS
  roscpp
  rospy
  std_msgs
  message_generation
)
```

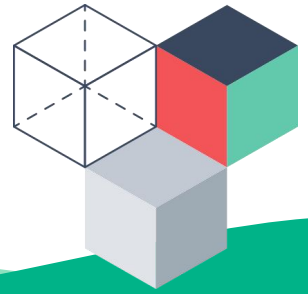
- 如此，所有 pacakge 里面的头文件和库文件等目录变成以 catkin_ 前缀的一组变量上，比如：catkin_INCLUDE_DIRS
- 这意味着 roscpp 导出的包含路径、库等也附加到 catkin_ 变量。 例如，catkin_INCLUDE_DIRS 不仅包含 catkin 的包含路径，还包含 roscpp 的包含路径



1.1.5 Enable Python module support (catkin_python_setup())

- 如果 catkin 包使用一些 Python 模块，应该创建一个 setup.py 文件，并在调用 catkin_python_setup() 之后调用 generate_messages() 和 catkin_package()

```
## Uncomment this if the package has a setup.py. This macro ensures
## modules and global scripts declared therein get installed
## See http://ros.org/doc/api/catkin/html/user_guide/setup_dot_py.html
# catkin_python_setup()
```



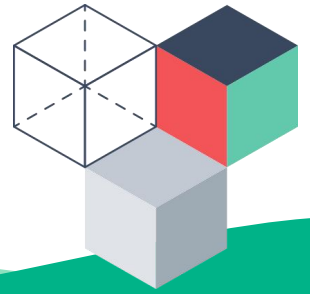
1.1 CMake

1.1.6 Message/Service/Action Generators (add_message_files(), add_service_files(), add_action_files())

- ROS 中的消息 (.msg)、服务 (.srv) 和操作 (.action) 文件在被 ROS 包构建和使用之前需要一个特殊的预处理器构建步骤，把其转化为系统可以识别特定编程语言（.h .cpp）
- 系统会用一些编程语言的生成器（例如 gencpp, genpy, genlisp, etc）生成相应的.cpp 和 .py文件。这个过程需要三个宏：add_message_files， add_service_files，add_action_files 来相应的控制 .msg .srv .action
- 这些宏后面必须跟着一个 generate_messages() 的宏，运用过程中要注意：

1. 这些宏必须在 catkin_package() 宏前面，即：

```
find_package(catkin REQUIRED COMPONENTS ...)
add_message_files(...)
add_service_files(...)
add_action_files(...)
generate_messages(...)
catkin_package(...)
...
```



1.1 CMake

1.1.6 Message/Service/Action Generators (add_message_files(), add_service_files(), add_action_files())

```
##############################################
## Declare ROS messages, services and actions ##
##############################################

## To declare and build messages, services or actions from within this
## package, follow these steps:
## * Let MSG_DEP_SET be the set of packages whose message types you use in
##   your messages/services/actions (e.g. std_msgs, actionlib_msgs, ...).
## * In the file package.xml:
##   * add a build_depend tag for "message_generation"
##   * add a build_depend and a exec_depend tag for each package in MSG_DEP_SET
##   * If MSG_DEP_SET isn't empty the following dependency has been pulled in
##     but can be declared for certainty nonetheless:
##     * add a exec_depend tag for "message_runtime"
## * In this file (CMakeLists.txt):
##   * add "message_generation" and every package in MSG_DEP_SET to
##     find_package(catkin REQUIRED COMPONENTS ...)
##   * add "message_runtime" and every package in MSG_DEP_SET to
##     catkin_package(CATKIN_DEPENDS ...)
##   * uncomment the add_*_files sections below as needed
##     and list every .msg/.srv/.action file to be processed
##   * uncomment the generate_messages entry below
##   * add every package in MSG_DEP_SET to generate_messages(DEPENDENCIES ...)
```

```
## Generate messages in the 'msg' folder
add_message_files(
  FILES
#  Message1.msg
#  Message2.msg
  test.msg
)

## Generate services in the 'srv' folder
# add_service_files(
#   FILES
#   Service1.srv
#   Service2.srv
# )

## Generate actions in the 'action' folder
# add_action_files(
#   FILES
#   Action1.action
#   Action2.action
# )

## Generate added messages and services with any dependencies listed here
generate_messages(
  DEPENDENCIES
  std_msgs
# )
```

①

②

③

④



1.1.6 Message/Service/Action Generators (add_message_files(), add_service_files(), add_action_files())

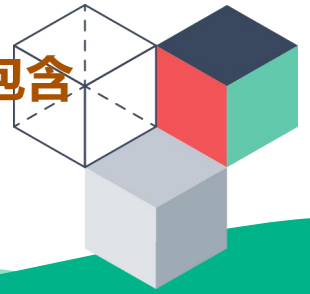
2. 宏 catkin_package() 中必须有 CATKIN_DEPENDS 依赖于 message_runtime，即

```
catkin_package(
 ...
 CATKIN_DEPENDS message_runtime ...
 ...)
```

3. find_package() 必须依赖包 message_generation

```
find_package(catkin REQUIRED COMPONENTS message_generation)
```

4. package.xml 文件 build_depend 必须包含 message_generation，run_depend 必须包含 message_runtime



1.1.6 Message/Service/Action Generators (add_message_files(), add_service_files(), add_action_files())

5. 如果构建的对象依赖于其他需要构建消息/服务/响应的构建对象，则需要向目标 catkin_EXPORTED_TARGETS 添加明确的依赖关系，以便它们以正确的顺序构建：

   add_dependencies(some_target ${catkin_EXPORTED_TARGETS})

   其中，some_target 是 add_executable() 设置的 target 的名字

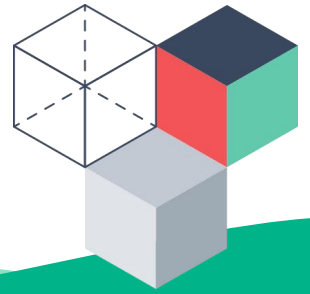
6. 如果需要构建消息或服务的包以及使用这些消息和/或服务的可执行文件，则需要为自动生成的消息目标创建明确的依赖关系，以便以正确的顺序构建它们：

   add_dependencies(some_target ${${PROJECT_NAME}_EXPORTED_TARGETS})

   其中，some_target 是 add_executable() 设置的 target 的名字

7. 如果你的包同时满足上述两个条件，则需要添加两个依赖项，即：

   add_dependencies(some_target ${${PROJECT_NAME}_EXPORTED_TARGETS}
   ${catkin_EXPORTED_TARGETS})



1.1 CMake

1.1.6 Message/Service/Action Generators (add_message_files(), add_service_files(), add_action_files())

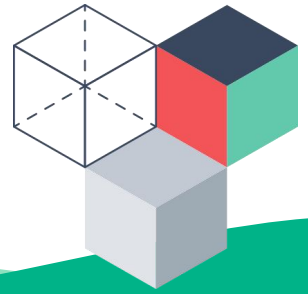
- 如果要构建 actionlib，则在功能包中“action”目录中创建有一个名为“MyAction.action”的文件，必须将 actionlib_msgs 添加到使用 catkin 进行 find_package 的组件列表中，然后在CMakeList.txt中加上下面两行：

```
add_action_files(FILES

  MyAction.action

)
```

- 此外，该包必须对 actionlib_msgs 具有构建依赖性



1.1 CMake

1.1.6 Message/Service/Action Generators (add_message_files(), add_service_files(), add_action_files())

➢ 举例：

a) 现在有两个依赖于 std_msgs 和 sensor_msgs 的消息 MyMessage1.msg和 MyMessage2.msg，还有一个自定义服务 MyService.srv

b) message_program 是使用这些消息和服务的命令，以及生成不使用自定义消息、服务的程序 do_not_use_local_messages_program的catkin包

c) 那么 CMakeLists.txt 应该写成：

切换行号显示

```
# Get the information about this package's buildtime dependencies
find_package(catkin REQUIRED
  COMPONENTS message_generation std_msgs sensor_msgs)

# Declare the message files to be built
add_message_files(FILES
  MyMessage1.msg
  MyMessage2.msg
)

# Declare the service files to be built
add_service_files(FILES
  MyService.srv
)

# Actually generate the language-specific message and service files
generate_messages(DEPENDENCIES std_msgs sensor_msgs)

# Declare that this catkin package's runtime dependencies
catkin_package(
 CATKIN_DEPENDS message_runtime std_msgs sensor_msgs
)

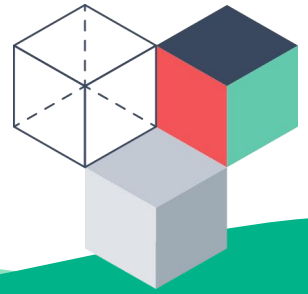
# define executable using MyMessage1 etc.
add_executable(message_program src/main.cpp)
add_dependencies(message_program ${${PROJECT_NAME}_EXPORTED_TARGETS} ${catkin_EXPORTED_TARGETS})

# define executable not using any messages/services provided by this package
add_executable(does_not_use_local_messages_program src/main.cpp)
add_dependencies(does_not_use_local_messages_program ${catkin_EXPORTED_TARGETS})
```

1.1 CMake

1.1.7 Specify package build info export (catkin_package())

- catkin_package() 是 catkin 提供的 CMake 宏，用于将 catkin 特定的信息输出到构建系统上，用于生成 pkg-config 和 CMake 文件
- 这个命令必须在 add_library() 或者 add_executable() 之前调用，该函数有5个可选参数：
  - INCLUDE_DIRS - 导出包的 include 路径
  - LIBRARIES - 导出项目中的库
  - CATKIN_DEPENDS - 该项目依赖的其他 catkin 项目
  - DEPENDS - 该项目所依赖的非 catkin CMake 项目
  - CFG_EXTRAS - 其他配置选项



1.1 CMake

1.1.7 Specify package build info export (catkin_package())

- 举例：

```
catkin_package(
  INCLUDE_DIRS include
  LIBRARIES ${PROJECT_NAME}
  CATKIN_DEPENDS roscpp rospy
  DEPENDS eigen opencv)
```

- 这表示工作空间中的“include”文件夹是导出头文件的地方； ${PROJECT_NAME}根据 project中的内容生成，此处是 beginner_tutorials；“roscpp”+“rospy”是用来构建/运行此程序包的 catkin 包，而“eigen”+“opencv”是用于构建/运行此程序包的非 catkin 包



1.1.8 Libraries/Executables to build (add_library()/add_executable()/target_link_libraries())

➢ 构建目标可以有多种形式，但通常主要有以下两种：

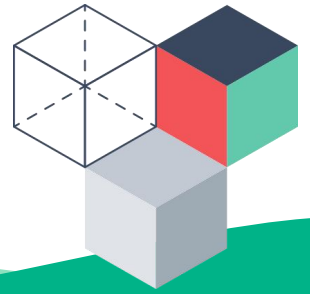
- 执行文件目标 - 可以运行的程序
- 库目标 - 可在构建或运行时给可执行文件使用的库

1. target 的命名：

- target 的命名很重要，在 catkin 中 target 的名字必须是唯一的，和之前构建产生的和安装的都不能相同
- 可以利用 set_target_properties() 函数将 target 进行重命名。例如：

```
set_target_properties(rviz_image_view
                      PROPERTIES OUTPUT_NAME image_view
                      PREFIX "")
```

上述操作将 rviz_image_view 的名称更改为image_view



1.1.8 Libraries/Executables to build (add_library()/add_executable()/target_link_libraries())

2. 自定义输出路径：

- ROS中的输出路径时默认的，但是也可通过下面进行修改：

```
set_target_properties(python_module_library

  PROPERTIES LIBRARY_OUTPUT_DIRECTORY
${CATKIN_DEVEL_PREFIX}/${CATKIN_PACKAGE_PYTHON_DESTINATION})
```

3. 头文件路径和库文件路径：

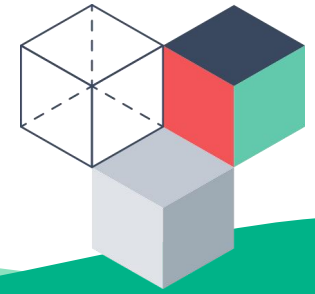
- 在我们指定 target 之前，需要指明相对 target 而言，在哪里找源文件，特别是在哪里找头文件，在哪里找库文件：

Include Paths：指明编译代码时在哪里寻找头文件

Library Paths：指明可执行文件需要的库文件在哪里

include_directories(<dir1>, <dir2>, ..., <dirN>)

link_directories(<dir1>, <dir2>, ..., <dirN>)



1.1.8 Libraries/Executables to build (add_library()/add_executable()/target_link_libraries())

3. 头文件路径和库文件路径：

- 函数 include_directories()：

  include_directories() 中的参数是调用 find_package 时生成的 * _INCLUDE_DIRS 变量。如果使用 catkin 和 Boost，那么 include_directories() 调用应该如下所示：

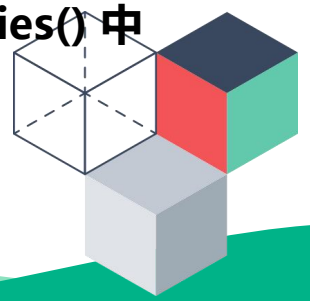
  include_directories(include ${Boost_INCLUDE_DIRS} ${catkin_INCLUDE_DIRS})

  第一个参数“include”表示功能包中的include /目录也是路径的一部分

- 函数 link_directories()：

  link_directories() 函数可用于添加额外的库路径，但通常不推荐这样做，因为所有 catkin 和 CMake 软件包在 find_packaged 时都会自动添加链接信息。 只需写 target_link_libraries() 中就可以了。若要使用，采用以下方式：

  link_directories(~/my_libs)



1.1.8 Libraries/Executables to build (add_library()/add_executable()/target_link_libraries())

4. 可执行目标：

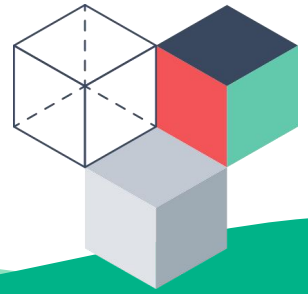
- 要指定必须构建的可执行目标，使用 add_executable() 函数：

```
add_executable(myProgram src/main.cpp src/some_file.cpp src/another_file.cpp)
```

上述操作这将构建一个名为 myProgram 的目标可执行文件，它由 3 个源文件构建：src/main.cpp、src/some_file.cpp 和 src/another_file.cpp

5. 库目标：

```
add_library(${PROJECT_NAME} ${${PROJECT_NAME}_SRCS})
```



1.1.8 Libraries/Executables to build (add_library()/add_executable()/target_link_libraries())

6. 链接库 target_link_libraries:

- target_link_libraries() 用来指定可执行目标链接的库。 通常在 add_executable() 调用之后完成。如果找不到 ros，则添加${catkin_LIBRARIES}:

  target_link_libraries(<executableTargetName>, <lib1>, <lib2>, ... <libN>)

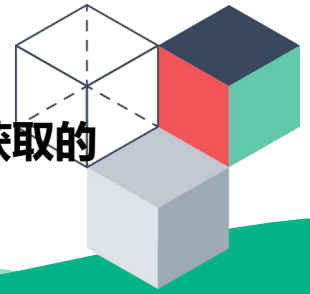
- 举例:

```
add_executable(foo src/foo.cpp)

add_library(moo src/moo.cpp)

target_link_libraries(foo moo)
```

- 注意：在大多数用例中不需要使用 link_directories()，因为该信息是通过 find_package() 自动获取的



Summary

1. CMakeLists.txt Introduction

a) Required CMake Version (cmake_minimum_required)

b) Package Name (project())

c) Find other CMake/Catkin packages needed for build (find_package())

d) Enable Python module support (catkin_python_setup())

e) Message/Service/Action Generators (add_message_files(), add_service_files(), add_action_files())

f) Invoke message/service/action generation (generate_messages())

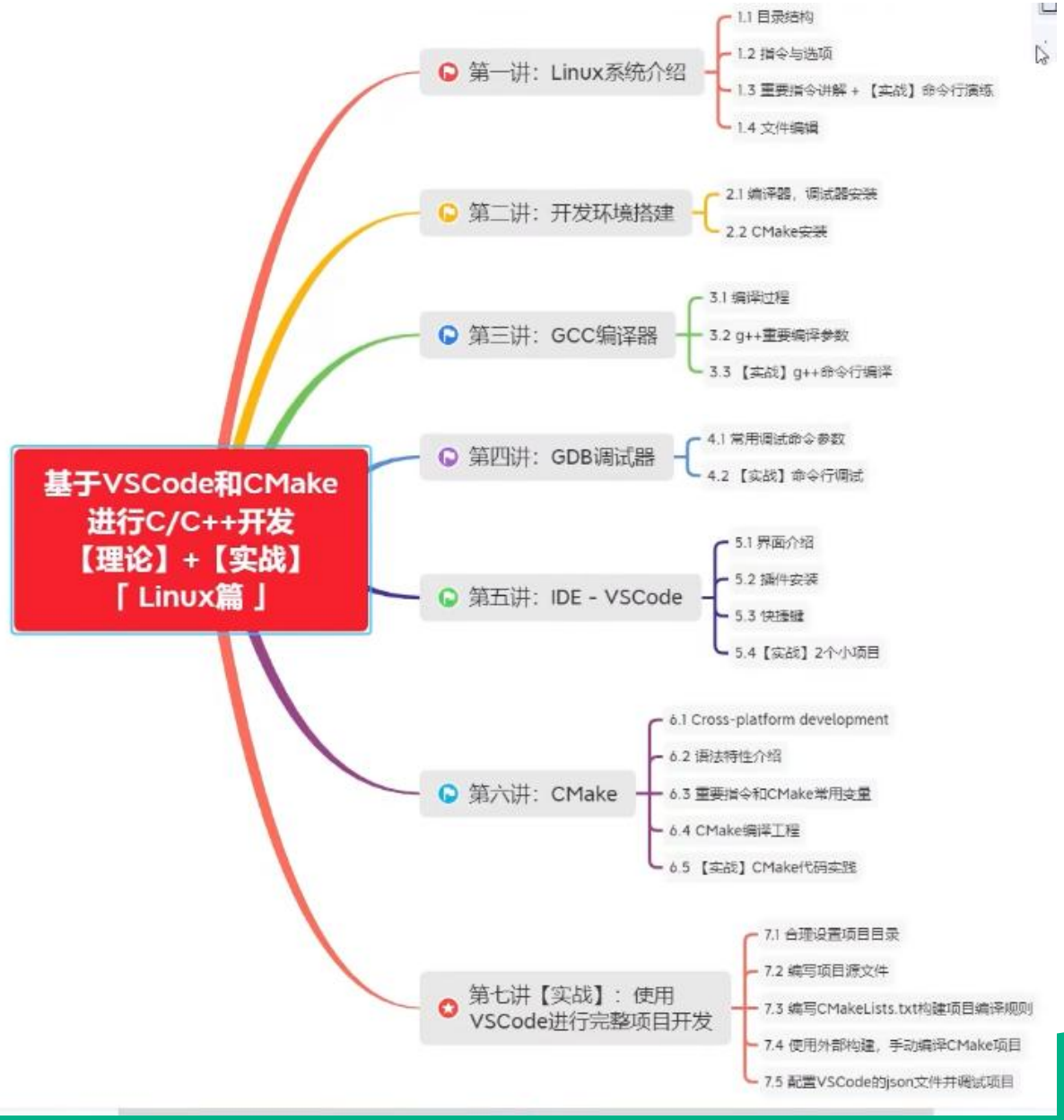
g) Specify package build info export (catkin_package())

h) Libraries/Executables to build (add_library()/add_executable()/target_link_libraries())

Homework

1 基于VSCode和CMake实现C/C++开发 | Linux篇

https://www.bilibili.com/video/BV1fy4y1b7TC/?spm_id_from=333.999.0.0&vd_source=7df640296c15e75b5fbd2da8cc72a434



2. 作业内容：

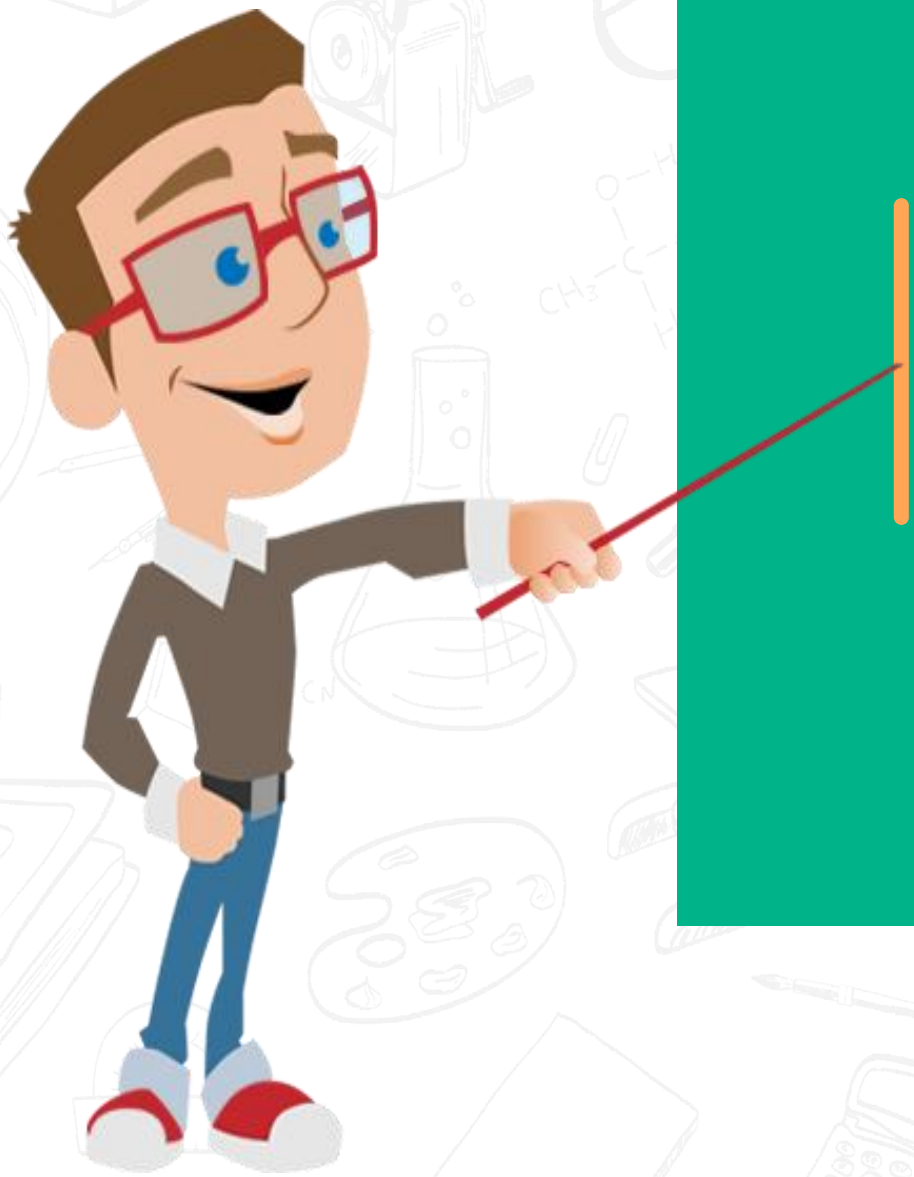
a) 学习课程所有内容，并跟着进行代码练习

b) 记录学习内容+测试代码

Q&A

今日事今日毕

Question?



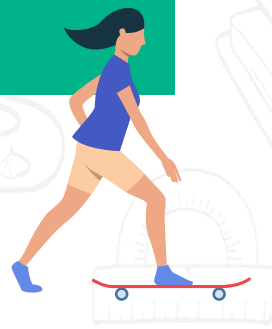
机器人操作系统（双语）

Robot Operating System (Bilingual)

WHERE THERE'S A WILL, THERE'S A WAY

授课教师：郄(qiè)龙飞

机器人工程



Overview lesson 7-2

1. ROS Computation Graph
2. ~
3. ~
4. ~

Objectives

1、掌握 ROS Computation Graph 的基本概念

2、熟练使用 ROS Computation Graph 命令行工具



Pre-assessment

1、Computation Graph 包含哪些概念?

2、是否知晓如何使用 Computation Graph 工具?



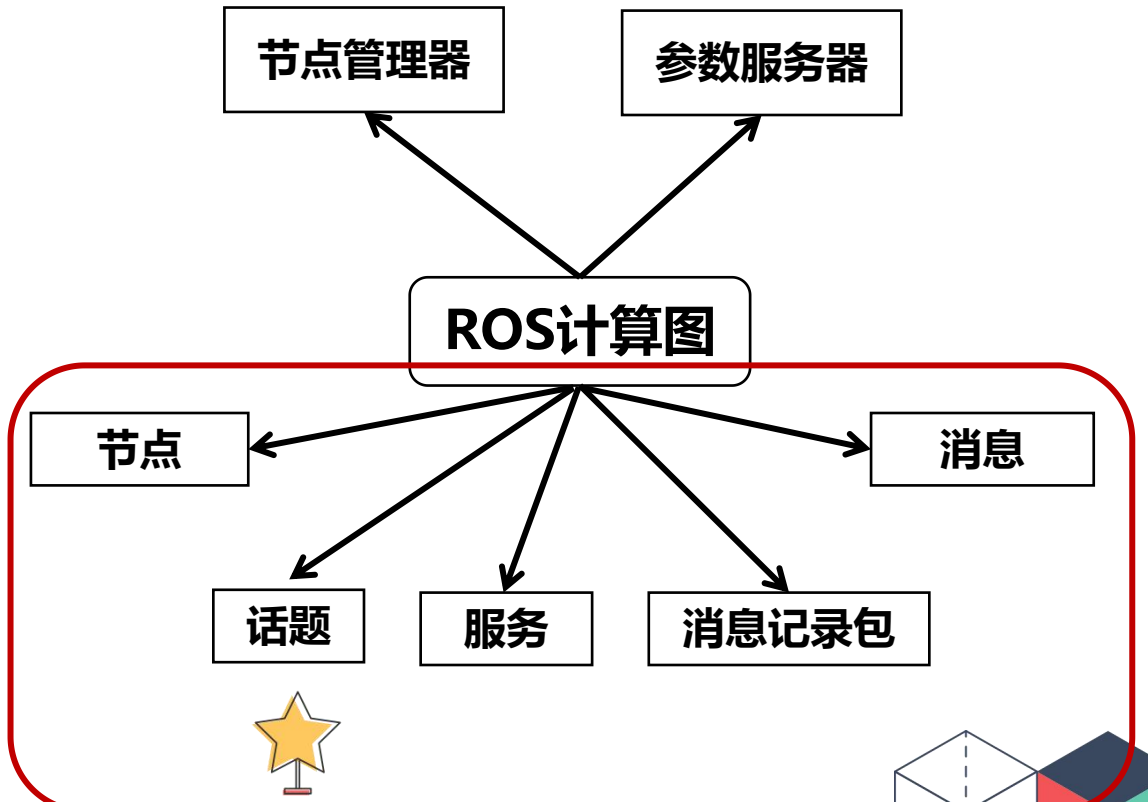
1

ROS Computation Graph

1.1 ROS computation graph level

1.1.1 Overview：http://wiki.ros.org/rosnode

- ROS会创建一个连接到所有进程的网络。系统中的任何节点都可以访问这个网络，并且通过该网络与其他节点进行交互(获取其他节点发布的消息+将自身数据发布到网络)
- 这个网络称为计算图，即计算图(Computation Graph)是ROS处理数据的一种点对点的网络形式
- 计算图中的基本元素是：节点(Node)、主机(ROS Master)、参数服务器(Parameter Server)、消息(Message)、服务(Service)、话题(Topics)和消息记录包(Bags)，它们都以不同的方式向 Graph 提供数据



1.1 ROS computation graph level

1.1.2 Basic concepts

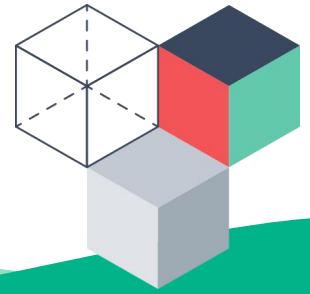
1. 节点(微信用户)：ROS节点对应于 Linux 系统中执行特定任务的进程

   - ROS节点通过 ROS 客户端库编写，例如 roscpp 或 rospy
   - ROS节点支持 ROS 消息和 ROS 服务两种通讯方式
   - ROS的设计理念是让多个节点协同工作，而不是让一个节点包揽所有任务。例如，一个机器人控制系统通常有很多节点组成，一个节点控制激光测距仪，一个节点执行定位，一个节点执行路径规划等

```
ros@ROS-VirtualBox:~$ rosnode --help
rosnode is a command-line tool for printing information about ROS Nodes.

Commands:
        rosnode ping    test connectivity to node
        rosnode list    list active nodes
        rosnode info    print information about node
        rosnode machine list nodes running on a particular machine or list machi
nes
        rosnode kill    kill a running node
        rosnode cleanup purge registration information of unreachable nodes

Type rosnode <command> -h for more detailed usage, e.g. 'rosnode ping -h'
```

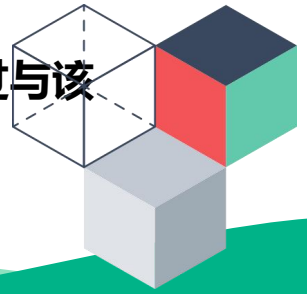


1.1 ROS computation graph level

1.1.2 Basic concepts

2. 主机(微信)：ROS Master 为节点提供名称注册和查找服务

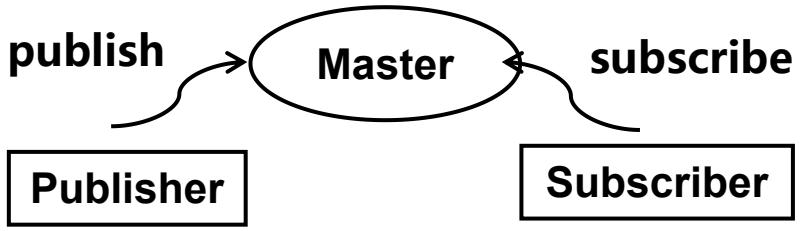
- ROS Master 负责在节点间建立连接，没有 ROS Master，节点将无法找到彼此，无法进行数据交换和服务调用
- 在 ROS 系统中，任何节点启动时会将节点名称注册到 ROS Master，因此 ROS Master 具有当前ROS系统上运行的所有节点信息
- 在节点发布话题之前，其将向 ROS Master 提供该话题的详细信息，包括其名称和数据类型等
- ROS Master 检查是否有其他节点订阅此话题：若有，则 ROS Master 将发布节点的细节共享给订阅节点，并通过 TCPROS 协议与发布节点进行互连，之后 ROS Master 放弃对节点的控制
- ROS 系统只能有一个 ROS Master，可以在一台计算机上运行 ROS Master，远程节点可以通过与该Master与其他节点进行通讯(需要定义ROS_MASTER_URI

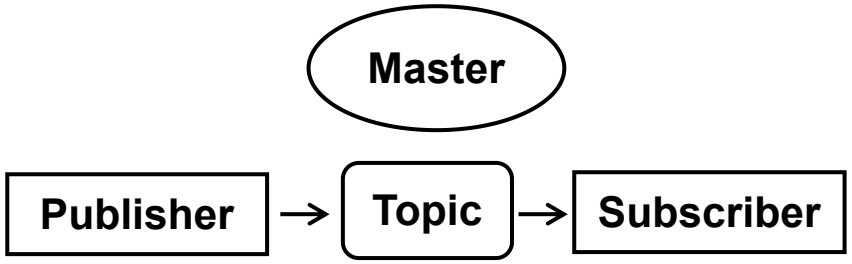


1.1 ROS computation graph level

1.1.2 Basic concepts

2. 主机(微信)：ROS Master 为计算图的其余部分提供名称注册和查找。没有 ROS Master，节点将无法找到彼此、交换消息或调用服务






```
... logging to /home/ros/.ros/log/bbdddf0e-f987-11eb-840c-94f6d62469e0/roslaunch-ros-mac-17802.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://ros-mac:41783/
ros_comm version 1.14.11


SUMMARY
========

PARAMETERS
 * /rosdistro: melodic
 * /rosversion: 1.14.11

NODES

auto-starting new master
process[master]: started with pid [17813]
ROS_MASTER_URI=http://ros-mac:11311/

setting /run_id to bbdddf0e-f987-11eb-840c-94f6d62469e0
process[rosout-1]: started with pid [17824]
started core service [/rosout]
```

1.1 ROS computation graph level

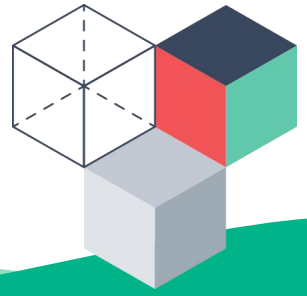
1.1.2 Basic concepts

3. 参数服务器(微信设置)：参数服务器是 ROS Master 的一部分

- 允许 ROS 将数据或者配置信息集中存储，所有节点都可以访问和修改这些参数值
- 参数服务器是一个共享多变量字典，节点可以使用该服务器在运行时存储和检索参数
- 参数服务器使用 XMLRPC 实现，XMLRPC 是一种 RPC 协议（RPC（Remote Procedure Call）远程过程调用，简单的理解是一个节点请求另一个节点提供的服务），XMLRPC 是一个使用 XML 编码并以 HTTP 作为传输机制的远程调用

```
ros@ROS-VirtualBox:~$ rosparam --help
rosparam is a command-line tool for getting, setting, and deleting parameters fr
om the ROS Parameter Server.

Commands:
      rosparam set    set parameter
      rosparam get    get parameter
      rosparam load   load parameters from file
      rosparam dump   dump parameters to file
      rosparam delete delete parameter
      rosparam list   list parameter names
```



1.1 ROS computation graph level

1.1.2 Basic concepts

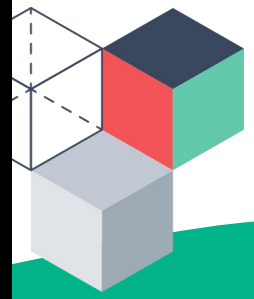
4. 消息(微信文字消息)：ROS 节点通过传递消息相互通信，应用于单向异步“发布-订阅”模式的通讯

- ROS节点通过消息互相通信，消息中包含该节点向其他节点提供信息的数据
- 消息是一种数据结构，具有标准的数据类型，以及基本类型数组，ROS提供一种基于标准消息类型开发自定义消息类型的机制
- 消息类型遵循以下命名约定：包的名称+.msg文件的名称。例如， std_msgs/msg/String.msg的消息类型是std_msgs/String

```
ros@ROS-VirtualBox:~$ rosmsg --help
rosmsg is a command-line tool for displaying information about ROS Message types.

Commands:
        rosmsg show     Show message description
        rosmsg info     Alias for rosmsg show
        rosmsg list     List all messages
        rosmsg md5      Display message md5sum
        rosmsg package  List messages in a package
        rosmsg packages List packages that contain messages

Type rosmsg <command> -h for more detailed usage
```



1.1 ROS computation graph level

1.1.2 Basic concepts

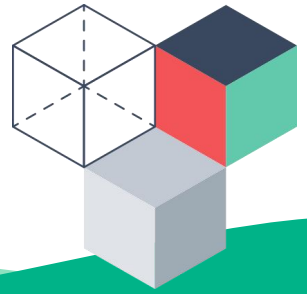
5. 服务(微信语音通话)：应用于双向同步“请求-相应”模式的节点通讯

- ROS Service 包含两个部分：请求 和 响应
- ROS Service 需要两个节点：服务端 和 客户端
- ROS Service 主要用于分布式系统中
- 服务类型遵循以下命名约定：包的名称+.srv文件的名称。例如， my_tutorials/srv/Test.srv的服务类型是my_tutorials/Test

```
ros@ROS-VirtualBox:~$ rossrv --help
rossrv is a command-line tool for displaying information about ROS Service types.

Commands:
        rossrv show     Show service description
        rossrv info     Alias for rossrv show
        rossrv list     List all services
        rossrv md5      Display service md5sum
        rossrv package  List services in a package
        rossrv packages List packages that contain services

Type rossrv <command> -h for more detailed usage
```



1.1 ROS computation graph level

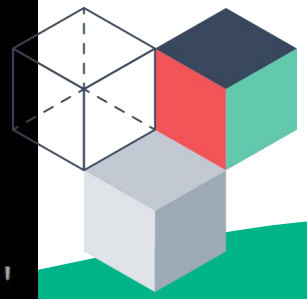
1.1.2 Basic concepts

5. 服务(微信语音通话)：应用于双向同步“请求-相应”模式的节点通讯

- 服务端节点通过名称提供服务，当客户端节点向服务端节点发送请求消息时，服务端节点响应这个消息并将结果发送到客户端节点
- 这个过程中，客户端节点需要一直等待，直到服务端节点响应客户端的请求
- rossrv 类似于 rosmsg，用于获取服务类型信息
- rosservice，用于列出服务列表和查询正在运动的 ROS 服务

```
ros@ROS-VirtualBox:~$ rosservice --help
Commands:
        rosservice args print service arguments
        rosservice call call the service with the provided args
        rosservice find find services by service type
        rosservice info print information about service
        rosservice list list active services
        rosservice type print service type
        rosservice uri  print service ROSRPC uri

Type rosservice <command> -h for more detailed usage, e.g. 'rosservice call -h'
```



1.1 ROS computation graph level

1.1.2 Basic concepts

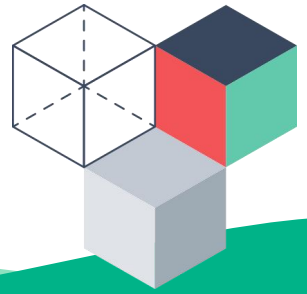
6. 话题(微信公众号)：遵循异步的发布-订阅机制，话题在节点之间通过消息交互信息

- 话题名称需具有唯一性
- 发布者/订阅者
- 发布者和订阅者相分离，允许多个发布者和多个订阅者，发布者和订阅者可以不了解彼此的存在
- 话题类型由发布在其上的消息类型决定，节点只能接受类型匹配的消息，一个节点只能订阅相同消息类型的话题

```
ros@ROS-VirtualBox:~$ rostopic --help
rostopic is a command-line tool for printing information about ROS Topics.

Commands:
        rostopic bw     display bandwidth used by topic
        rostopic delay  display delay of topic from timestamp in header
        rostopic echo   print messages to screen
        rostopic find   find topics by type
        rostopic hz     display publishing rate of topic
        rostopic info   print information about active topic
        rostopic list   list active topics
        rostopic pub    publish data to topic
        rostopic type   print topic or field type

Type rostopic <command> -h for more detailed usage, e.g. 'rostopic echo -h'
```



1.1 ROS computation graph level

1.1.2 Basic concepts

7. 消息记录包(微信数据存储)：用来保存和回放ROS消息数据的文件格式

- ROS bags 是一种用于存储数据的重要机制，其能够获取并记录各种传感器的数据
- 可以通过消息记录包反复获取实验数据，用于算法的测试和开发

```
ros@ROS-VirtualBox:~$ rosbag --help
Usage: rosbag <subcommand> [options] [args]

A bag is a file format in ROS for storing ROS message data. The rosbag command can record, replay an
d manipulate bags.

Available subcommands:
   check        Determine whether a bag is playable in the current system, or if it can be migrated.
   compress     Compress one or more bag files.
   decompress   Decompress one or more bag files.
   decrypt      Decrypt one or more bag files.
   encrypt      Encrypt one or more bag files.
   filter       Filter the contents of the bag.
   fix          Repair the messages in a bag file so that it can be played in the current system.
   help
   info         Summarize the contents of one or more bag files.
   play         Play back the contents of one or more bag files in a time-synchronized fashion.
   record       Record a bag file with the contents of specified topics.
   reindex      Reindexes one or more bag files.

For additional information, see http://wiki.ros.org/rosbag
```

1.1 ROS computation graph level

1.1.3 Sum-up

- 计算图(Computation Graph)是ROS处理数据的一种点对点的网络形式
- ROS 的基本 Computation Graph 概念是节点(Nodes)、主机(Master)、参数服务器(Parameter Server)、消息(Messages)、服务(Services)、话题(Topics)和消息记录包(Bags)，它们都以不同的方式向 Graph 提供数据

1. 节点：节点主要执行计算处理
   - ROS 被设计为细粒度的模块化的系统：一个机器人控制系统通常有很多节点组成 。例如，一个节点控制激光测距仪，一个节点执行定位，一个节点执行路径规划，一个节点提供系统图形界面等
   - ROS 节点通过 ROS 客户端库 client library编写，例如 roscpp 或 rospy
2. 主机：ROS Master 为计算图的其余部分提供名称注册和查找。没有 ROS Master，节点将无法找到彼此、交换消息或调用服务



1.1 ROS computation graph level

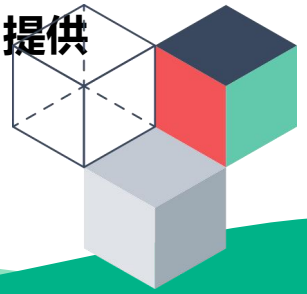
1.1.3 Sum-up

3. **参数服务器**：参数服务器允许将数据通过密钥存储在一个中央位置， 它目前是 ROS Master 的一部分

4. **消息**：ROS节点通过传递消息相互通信。消息是一种数据结构，支持标准基本类型（整数、浮点数、布尔值等），以及基本类型数组。 消息可以包含在任意嵌套的结构和数组中

5. **服务**：节点间的请求/响应的通讯模式

   - 发布/订阅模型是一种非常灵活的通信范式，但其多对多、单向传输不适用于请求/响应交互，而这在分布式系统中通常是必需的
   - 请求/响应是通过服务完成的，服务由一对消息结构定义：一个用于请求，一个用于响应。 提供节点以某个名称提供服务，客户端通过发送请求消息并等待回复来使用该服务



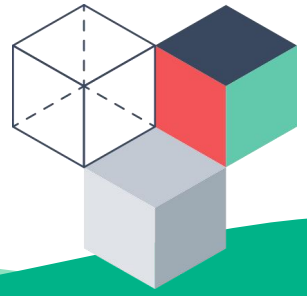
1.1 ROS computation graph level

1.1.3 Sum-up

6. **话题**：消息通过具有发布/订阅的传输系统进行通讯

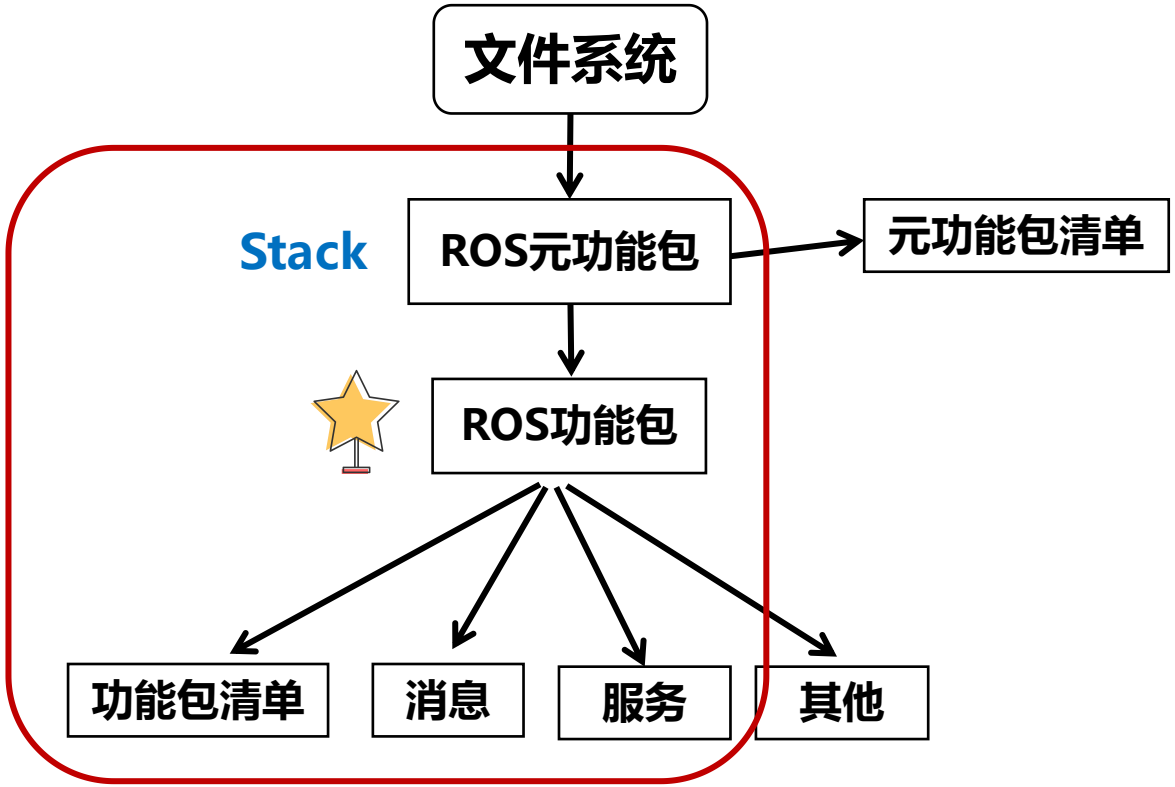
   - 节点通过将消息发布到给定主题来发送消息
   - 话题是用于标识消息内容的名称，对某种数据感兴趣的节点将订阅适当的话题
   - 一个话题可能有多个并发发布者和订阅者，一个节点可能发布和/或订阅多个话题

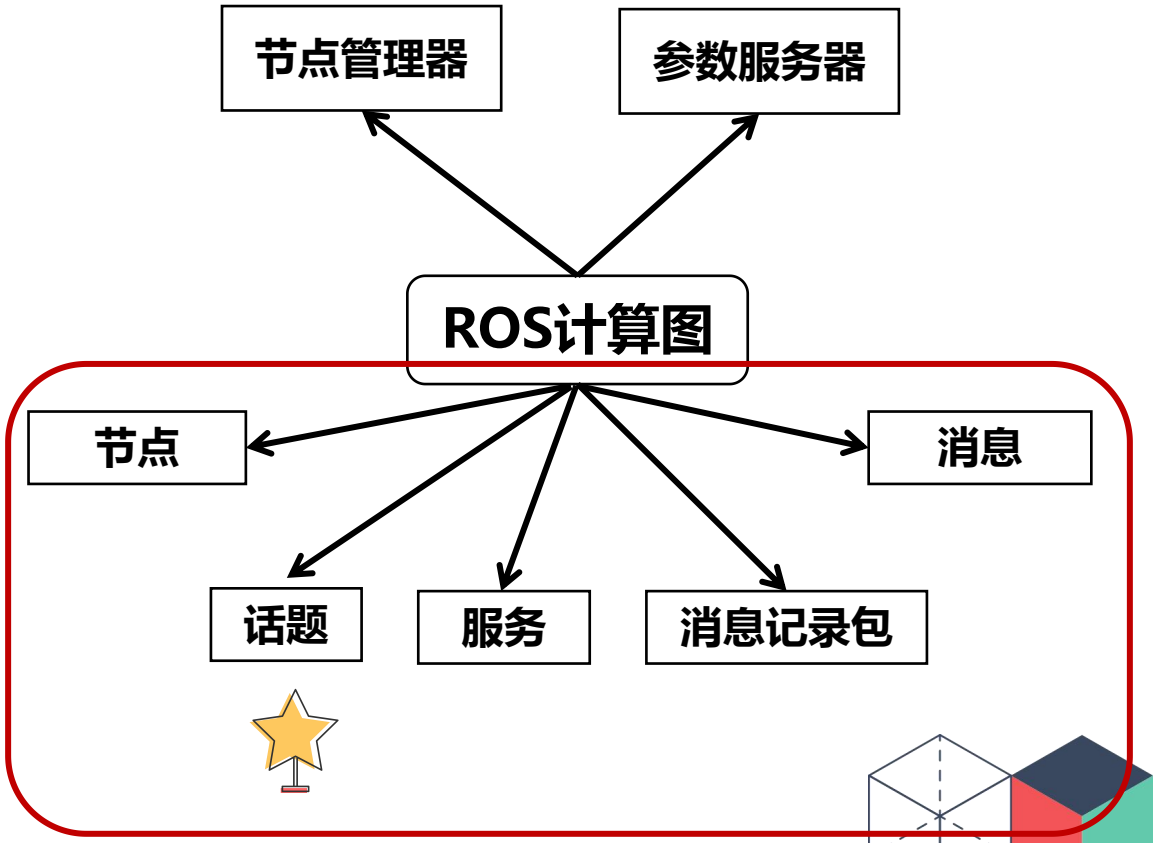
7. **消息记录包**：Bags 是一种用于保存和回放 ROS 消息数据的格式。Bags 是存储数据（例如传感器数据）的重要机制，这些数据可能难以收集，但却是开发和测试算法所必需的



1.1 ROS computation graph level

1.1.3 Sum-up

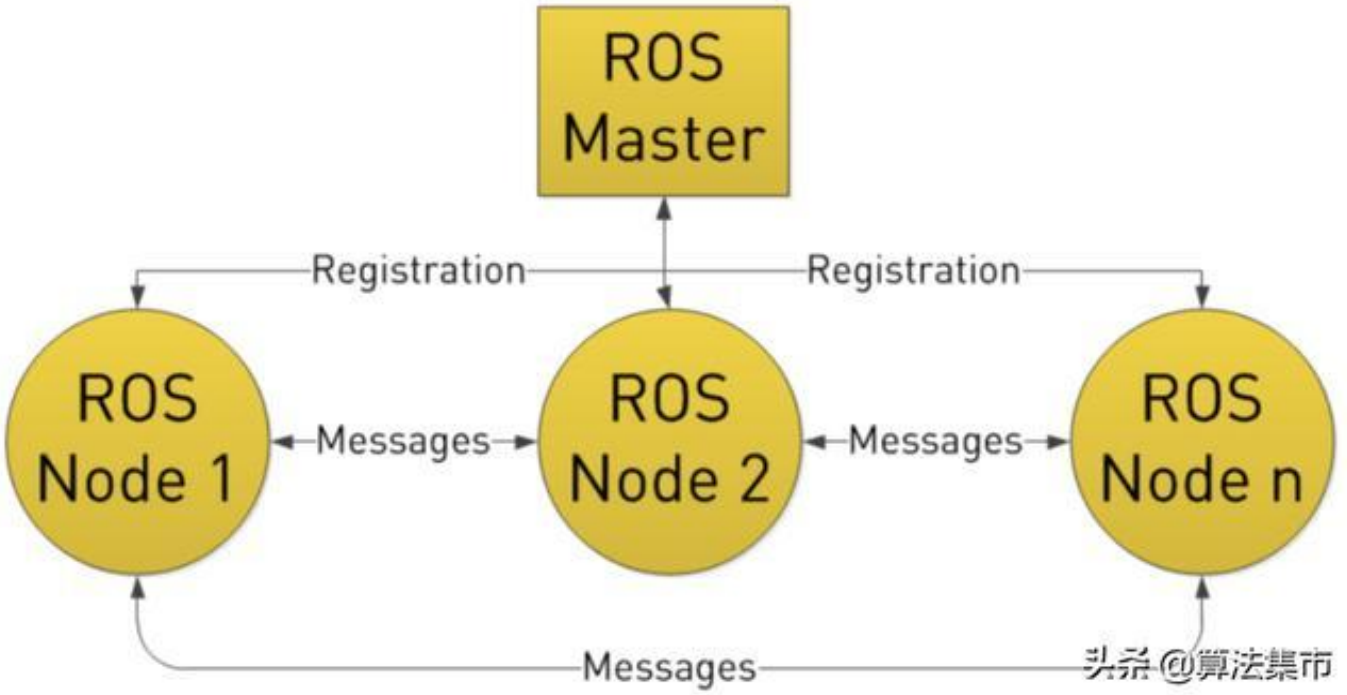




1.1 ROS computation graph level

1.1.4 Examples

- 节点之间通过消息交互信息

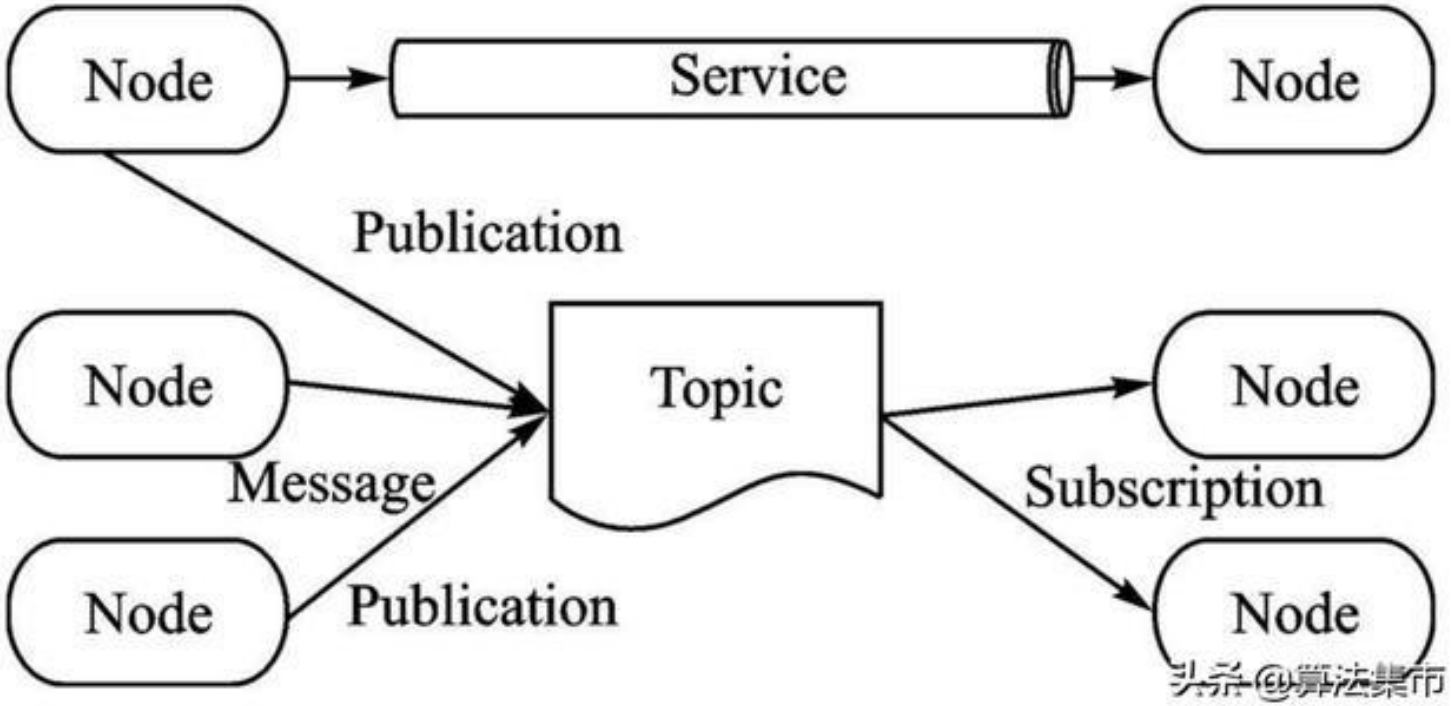


图片来源：算法集市

1.1 ROS computation graph level

1.1.4 Examples

- 异步“发布-订阅”模式

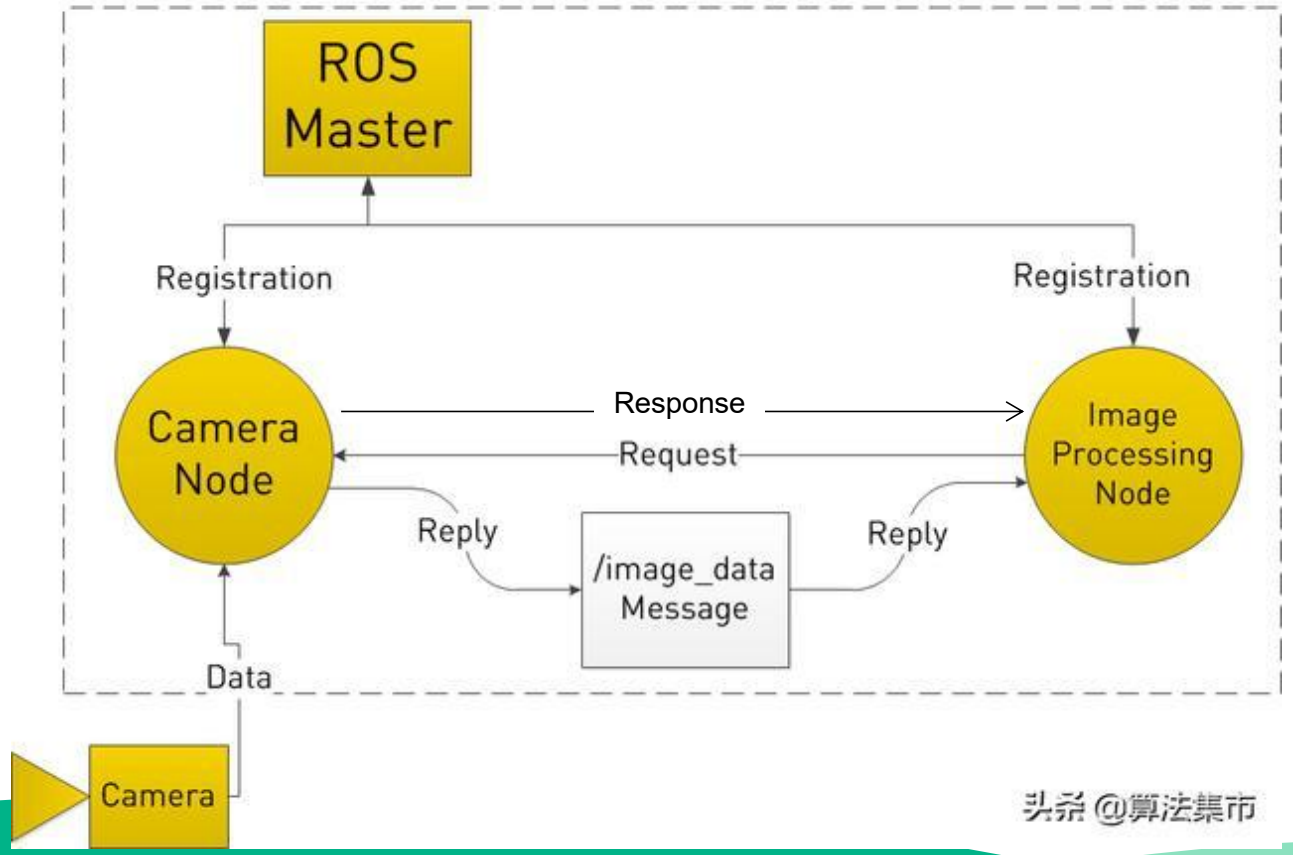


图片来源：算法集市

1.1 ROS computation graph level

1.1.4 Examples

- 同步“请求-响应”模式

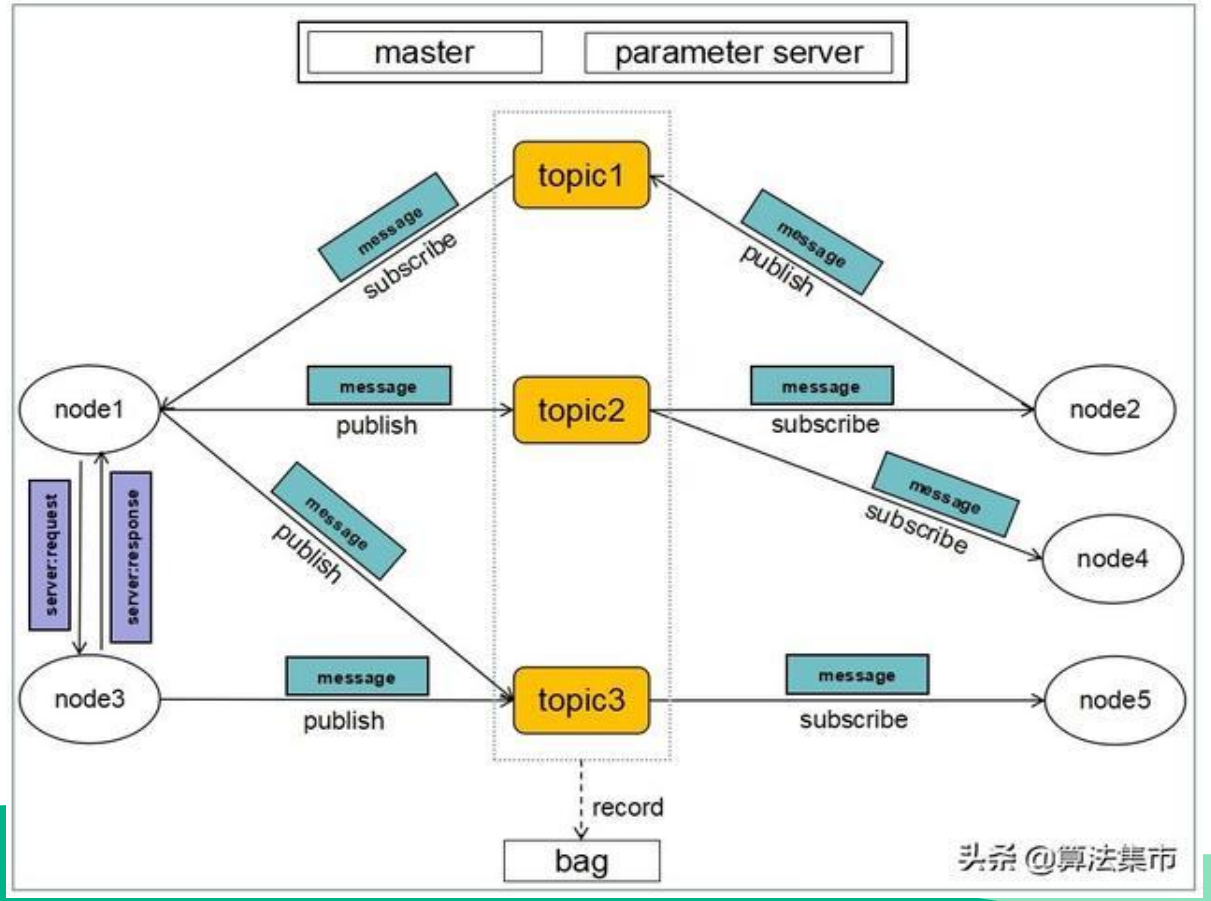


图片来源：算法集市

1.1 ROS computation graph level

1.1.4 Examples

- ROS中最小的进程单元就是节点（node），节点之间通过主题（topic）传递消息数据（message）



图片来源：算法集市

Summary

1. ROS Computation Graph Level

a) 节点 

b) 主机

c) 参数服务器

d) 话题 

e) 消息 

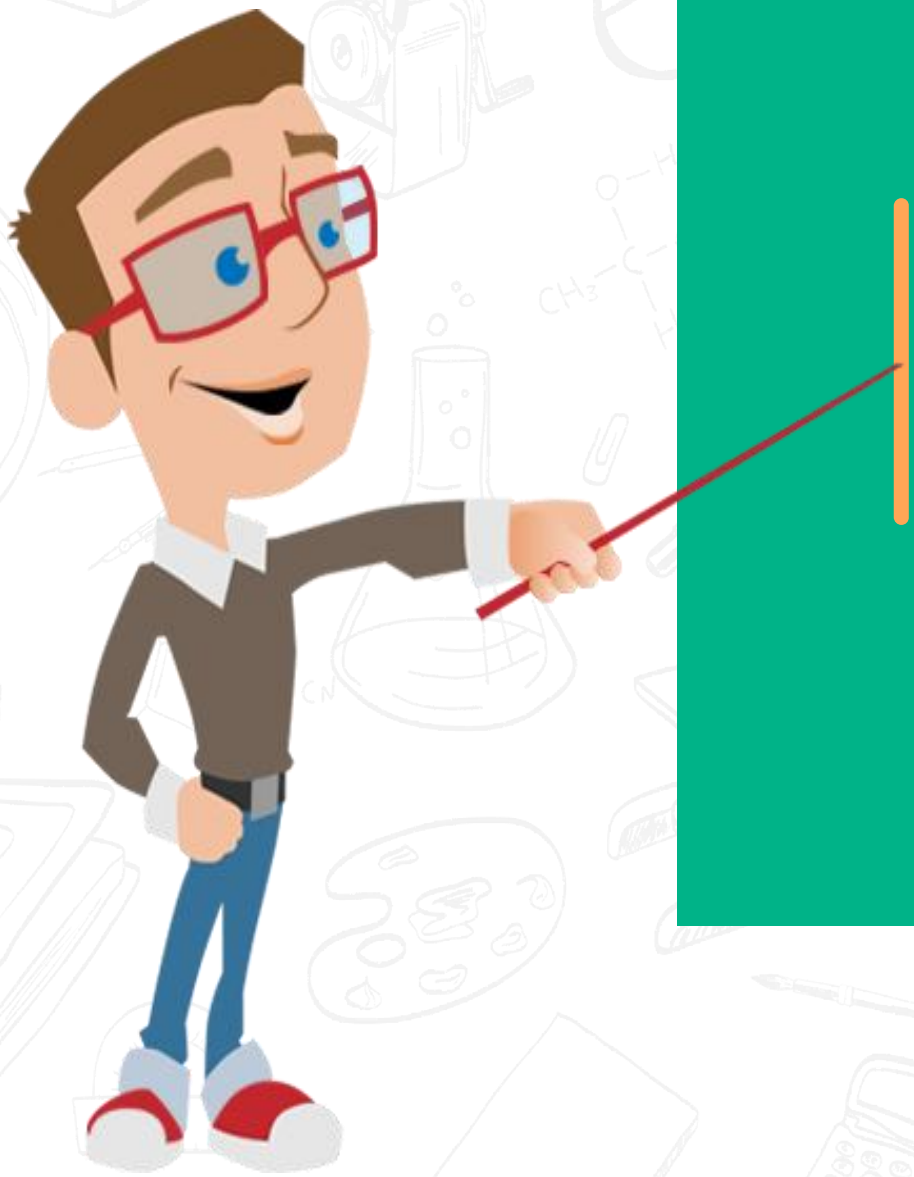
f) 服务 

g) 消息记录包 

Q&A

今日事今日毕

Question?



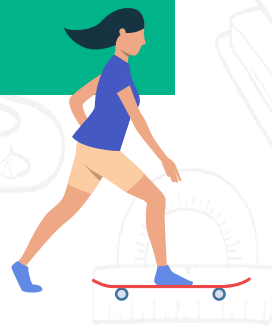
机器人操作系统（双语）

Robot Operating System (Bilingual)

WHERE THERE'S A WILL, THERE'S A WAY

授课教师：郄(qiè)龙飞

机器人工程



Overview lesson 7-3

1. Turtlesim
   Exercises
2. ~
3. ~
4. ~

Objectives

1、掌握节点、话题、消息、服务的使用方法

2、了解 rqt 使用方法

3、掌握 roslaunch 编写方法

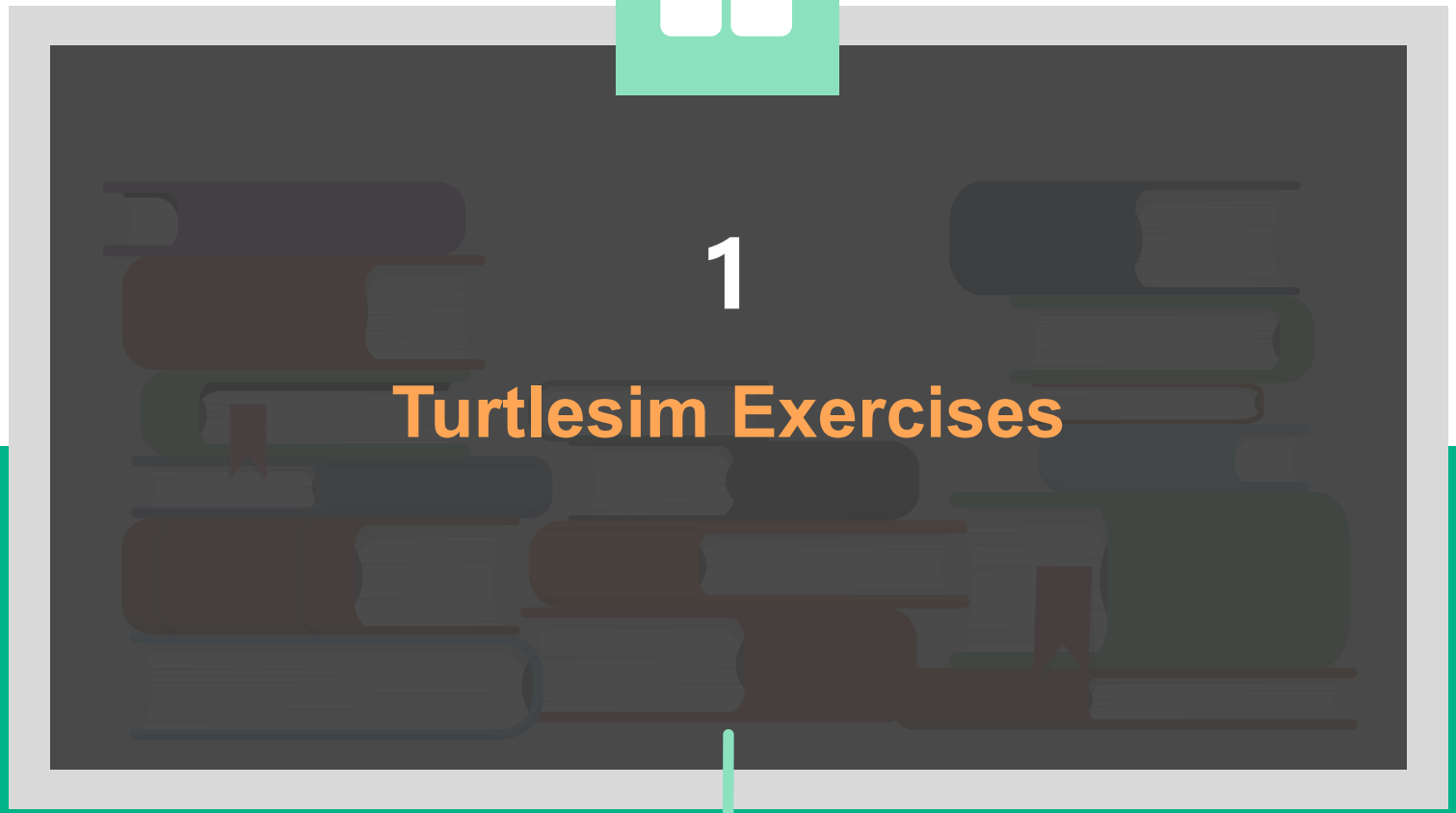


Pre-assessment

1、是否使用过turtlesim?

2、是否使用过rqt工具?

3、是否编写过roslaunch文件?



1

Turtlesim Exercises

1.1 Turtlesim Examples

1.1.1 Overview： http://wiki.ros.org/turtlesim

- **turtlesim 的4个节点：turtlesim_node，mimic，draw_square，turtle_teleop_key**

2. Nodes

New in ROS hydro As of *Hydro* turtlesim uses the geometry_msgs/Twist message (*turtlesim/Velocity* in *Groovy* and older). Also the topic has been changed to `cmd_ve` before).

2.1 turtlesim_node

`turtlesim_node` provides a simple simulator for teaching ROS concepts.

2.1.1 Subscribed Topics

`turtleX/cmd_vel` (geometry_msgs/Twist)

The linear and angular command velocity for turtleX. The turtle will execute a time out. Twist.linear.x is the forward velocity, Twist.linear.y is the strafe veloci velocity.

2.1.2 Published Topics

`turtleX/pose` (turtlesim/Pose)

The x, y, theta, linear velocity, and angular velocity of turtleX.

2.2 mimic

`mimic` provides a simple interface for making one turtlesim mimic another.

2.2.1 Subscribed Topics

`input` (turtlesim/Pose)

The input topic for the `mimic` node. The topic must be remapped to the `pos`

2.2.2 Published Topics

`output` (geometry_msgs/Twist)

The output topic for the `mimic` node. The topic must be remapped to the `cm`



1.1 Turtlesim Examples

1.1.1 Overview

- **turtlesim服务和参数：**

2.1.3 Services

`clear` (std_srvs/Empty)

Clears the turtlesim background and sets the color to the value of the backgrour

`reset` (std_srvs/Empty)

Resets the turtlesim to the start configuration and sets the background color to t

`kill` (turtlesim/Kill)

Kills a turtle by name.

`spawn` (turtlesim/Spawn)

Spawns a turtle at (x, y, theta) and returns the name of the turtle. Also will take r duplicate name.

`turtleX/set_pen` (turtlesim/SetPen)

Sets the pen's color (r g b), width (width), and turns the pen on and off (off).

`turtleX/teleport_absolute` (turtlesim/TeleportAbsolute)

Teleports the turtleX to (x, y, theta).

`turtleX/teleport_relative` (turtlesim/TeleportRelative)

Teleports the turtleX a linear and angular distance from the turtles current position.

2.1.4 Parameters

`~background_b` (`int`, default: 255)

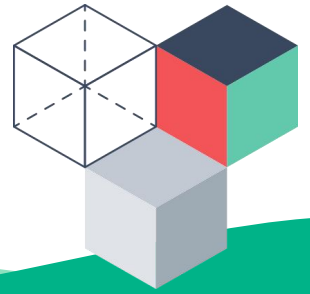
Sets the blue channel of the background color.

`~background_g` (`int`, default: 86)

Sets the green channel of the background color.

`~background_r` (`int`, default: 69)

Sets the red channel of the background color.



1.1 Turtlesim Examples

1.1.2 启动ros

- 命令行工具：

```
$ roscore
```

- 默认情况下，roscore 会启动三个程序：

1. ROS Master
2. Parameter Server
3. rosout 日志节点

```
ros@ROS-VirtualBox:~$ roscore
... logging to /home/ros/.ros/log/fa714a7e-30e3-11ec-bc7d-0
8002752f08a/roslaunch-ROS-VirtualBox-3520.log
Checking log directory for disk usage. This may take a whil
e.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.
```

主要用于从ROS节点来收集日志

```
started roslaunch server http://ROS-VirtualBox:46255/
ros_comm version 1.14.11
```

执行 roscore.xml 启动文件，启动ROS Master 和 Parameter Server

```
SUMMARY
========

PARAMETERS
 * /rosdistro: melodic
 * /rosversion: 1.14.11

NODES
```

roslaunch执行roscore.xml的显示结果

```
auto-starting new master
process[master]: started with pid [3531]
ROS_MASTER_URI=http://ROS-VirtualBox:11311/
```

Parameter Server的节点：ROS_MASTER_URI

```
setting /run_id to fa714a7e-30e3-11ec-bc7d-08002752f08a
process[rosout-1]: started with pid [3542]
started core service [/rosout]
```

ros节点启动，订阅/rosout话题并转播到/rosout_agg

1.1 Turtlesim Examples

1.1.2 启动ros

- 查看话题：

```
$ rostopic list
```

```
ros@ROS-VirtualBox:~$ rostopic list
/rosout
/rosout_agg
```

- 查看可用参数：

```
$ rosparam list
```

```
ros@ROS-VirtualBox:~$ rosparam list
/rosdistro
/roslaunch/uris/host_ros_virtualbox__46255
/rosversion
/run_id
```

/rosdistro ROS发行版

/roslaunch * roslaunch服务地址

/rosversion ROS版本号

/run_id ROS版本号

- 查看可用服务：

```
$ rosservice list
```

```
ros@ROS-VirtualBox:~$ rosservice list
/rosout/get_loggers
/rosout/set_logger_level
```

对应上述ros节点，用于设置日志目录

1.1 Turtlesim Examples

1.1.3 启动turtlesim

- **命令行：**

```
$ rosrun turtlesim turtlesim_node
```

- **rosrun命令：rosrun allows you to run an executable in an arbitrary package from anywhere without having to give its full path or cd/roscd there first.**

http://wiki.ros.org/rosbash#rosrun

Usage:

```
rosrun <package> <executable>
```

Example:

```
rosrun roscpp_tutorials talker
```

It's also possible to pass a `~parameter` using the following syntax (replace the ~ with an _):

```
rosrun package node _parameter:=value
```

Example:

```
rosrun my_package my_node _my_param:=value
```

...or run with a given name:

```
rosrun package node __name:=name
```

Example:

```
rosrun my_package my_node __name:=my_name
```

For more information about remapping, see: Remapping Arguments

Starting in **Indigo**, rosrun has a `--prefix` option which can be used to run a node in gdb or valgrind.

Example:

```
rosrun --prefix 'gdb -ex run --args' my_package my_node
```

1.1 Turtlesim Examples

1.1.3 启动turtlesim

- 运行结果：

```
ros@ROS-VirtualBox:~$ rosrun turtle
sim turtlesim_node
[ INFO] [1634659567.469690433]: Sta
rting turtlesim with node name /tur
tlesim
[ INFO] [1634659567.474676547]: Spa
wning turtle [turtle1] at x=[5.5444
45], y=[5.544445], theta=[0.000000]
```



- 查看话题：

```
$ rostopic list
```

```
ros@ROS-VirtualBox:~$ rostopic list
/rosout
/rosout_agg
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
```

- 查看节点：

```
$ rosnode list
```

```
ros@ROS-VirtualBox:~$ rosnode list
/rosout
/turtlesim
```

1.1 Turtlesim Examples

1.1.4 turtlesim节点和话题

- 查看/turtlesim节点详细信息：

```
$ rosnode info /turtlesim
```

```
ros@ROS-VirtualBox:~$ rosnode info /turtlesim
--------------------------------------------------------------------------------
Node [/turtlesim]
Publications:
 * /rosout [rosgraph_msgs/Log]
 * /turtle1/color_sensor [turtlesim/Color]
 * /turtle1/pose [turtlesim/Pose]

Subscriptions:
 * /turtle1/cmd_vel [unknown type]

Services:
 * /clear
 * /kill
 * /reset
 * /spawn
 * /turtle1/set_pen
 * /turtle1/teleport_absolute
 * /turtle1/teleport_relative
 * /turtlesim/get_loggers
 * /turtlesim/set_logger_level


contacting node http://ROS-VirtualBox:35153/ ...
Pid: 4355
Connections:
 * topic: /rosout
    * to: /rosout
    * direction: outbound (46185 - 127.0.0.1:56526) [26]
    * transport: TCPROS
```

1.1 Turtlesim Examples

1.1.4 turtlesim节点和话题

- 启动 turtle_teleop_key 节点：

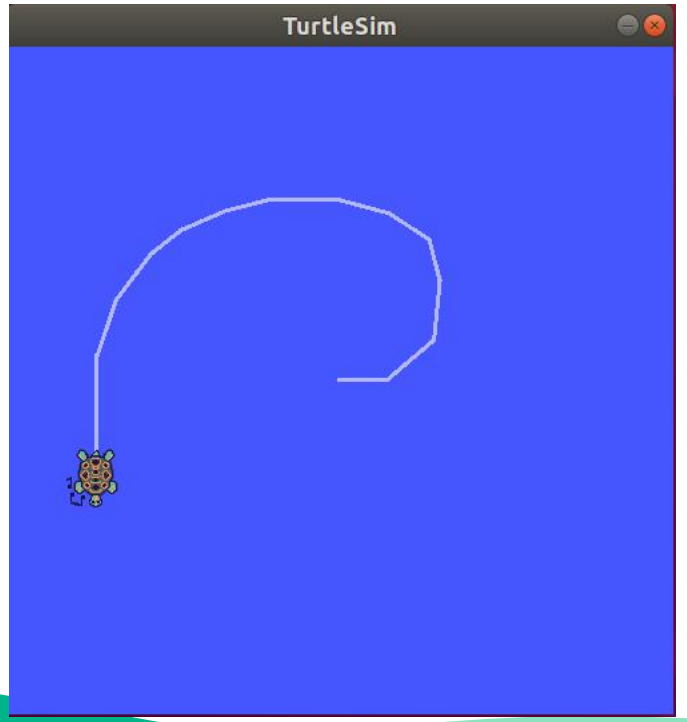
```
$ rosrun turtlesim turtle_teleop_key
```

```
ros@ROS-VirtualBox:~$ rosrun turtlesim
turtle_teleop_key
Reading from keyboard
---------------------------
Use arrow keys to move the turtle. 'q'
to quit.
```

- 查看ROS节点：

```
$ rosnode list
```

```
turtlesim/cmake$ rosnode list
/rosout
/teleop_turtle
/turtlesim
```



为什么操作键盘小乌龟会移动?

主观题 10分

设置

为什么操作键盘小乌龟会移动?

正常使用主观题需2.0以上版本雨课堂

作答

1.1 Turtlesim Examples

1.1.4 turtlesim节点和话题

- 查看节点和话题之间的关系：

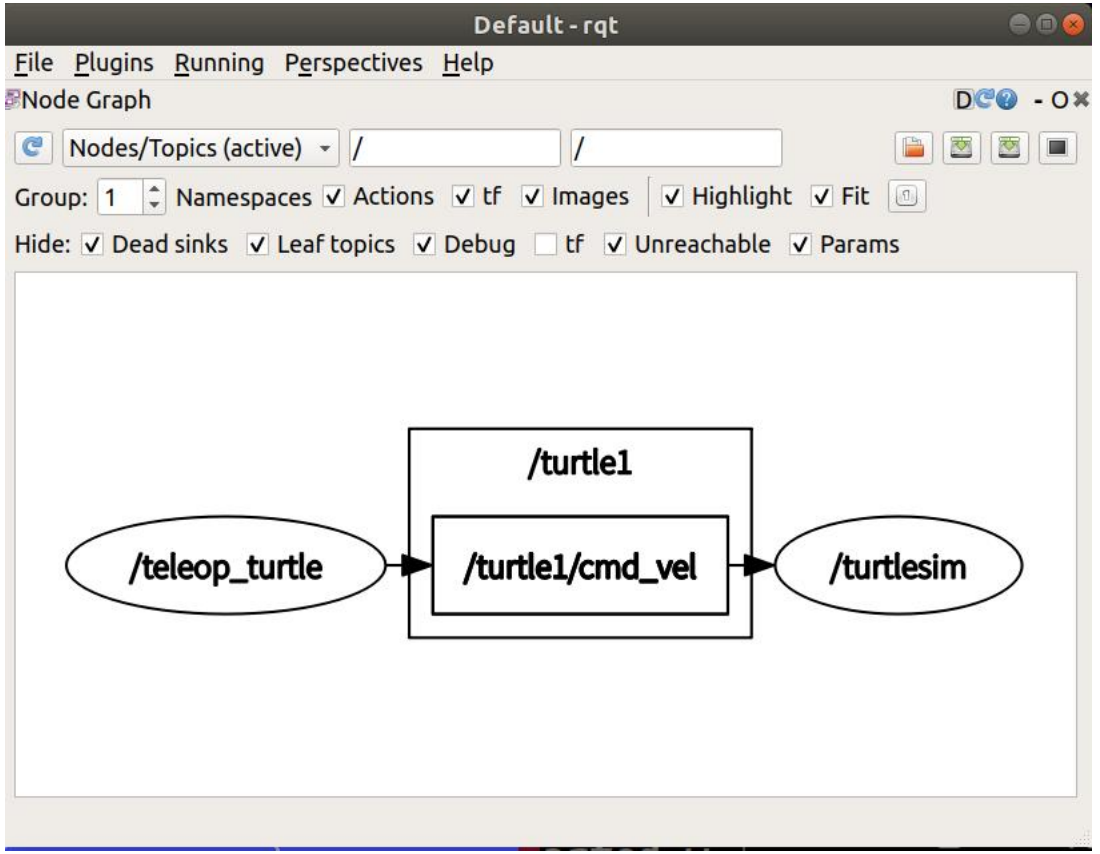
```
$ rosrun rqt_graph rqt_graph
```

```
$ rosrun rqt_graph rqt_graph
```

```
$ rqt
```

- rqt : rqt is a Qt-based framework for GUI development for ROS

https://wiki.ros.org/rqt



椭圆形表示节点，方框表示话题

1.1 Turtlesim Examples

1.1.4 turtlesim节点和话题

- 查看节点/teleop_turtle发送的数据：

```
$ rostopic echo /teleop_turtle
```

- 表明：节点 /teleop_turtle 正在发布节点 /turtlesim 订阅的话题

```
ros@ROS-VirtualBox:~$ rosnode info /teleop_turtle
--------------------------------------------------------------------------------
Node [/teleop_turtle]
Publications:
 * /rosout [rosgraph_msgs/Log]
 * /turtle1/cmd_vel [geometry_msgs/Twist]

Subscriptions: None

Services:
 * /teleop_turtle/get_loggers
 * /teleop_turtle/set_logger_level


contacting node http://ROS-VirtualBox:41265/ ...
Pid: 6387
Connections:
 * topic: /rosout
    * to: /rosout
    * direction: outbound (47331 - 127.0.0.1:48044) [12]
    * transport: TCPROS
 * topic: /turtle1/cmd_vel
    * to: /turtlesim
    * direction: outbound (47331 - 127.0.0.1:48046) [10]
    * transport: TCPROS
```

1.1 Turtlesim Examples

1.1.4 turtlesim节点和话题

- 查看话题 /turtle1/cmd_vel上的消息：

```
$ rostopic echo /turtle1/cmd_vel
```

```
ros@ROS-VirtualBox:~$ rostopic echo
 /turtle1/cmd_vel
linear:
  x: 2.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
```

- 查看话题类型(消息类型)：

```
$ rostopic type /turtle1/cmd_vel
```

```
ros@ROS-VirtualBox:~$ rostopic type
 /turtle1/cmd_vel
geometry_msgs/Twist
```

- 查看消息字段：

```
$ rosmsg info geometry_msgs/Twist
```

```
ros@ROS-VirtualBox:~$ rosmsg info
geometry_msgs/Twist
geometry_msgs/Vector3 linear
  float64 x
  float64 y
  float64 z
geometry_msgs/Vector3 angular
  float64 x
  float64 y
  float64 z
```

1.1 Turtlesim Examples

1.1.4 turtlesim节点和话题

- rostopic pub语法结构：

  rostopic pub /topic msg_type [args...]

- 手动向话题 /turtle1/cmd_vel上发布消息：

```
$ rostopic pub /turtle1/cmd_vel geometry_msgs/Twist [5,0,0] [0,0,0]
```

发布一次

```
$ rostopic pub /turtle1/cmd_vel geometry_msgs/Twist -r 1  [1,0,0] [0,0,1]
```

1秒一次的速率循环发布

```
ros@ROS-VirtualBox:~$ rostopic pub
 /turtle1/cmd_vel geometry_msgs/Tw
ist [5,4,0] [0,0,0]
publishing and latching message. P
ress ctrl-C to terminate
```

```
ros@ROS-VirtualBox:~$ rostopic pub /tu
rtle1/cmd_vel geometry_msgs/Twist -r 1
  [1,0,0] [0,0,1]
```

1.1 Turtlesim Examples

1.1.5 turtlesim服务

- 查询节点/turtlesim可用的服务：

$ rosservice list

```
ros@ROS-VirtualBox:~$ rosservice list
/clear
/kill
/reset
/rosout/get_loggers
/rosout/set_logger_level
/rostopic_7623_1634663943840/get_loggers
/rostopic_7623_1634663943840/set_logger_level
/spawn
/teleop_turtle/get_loggers
/teleop_turtle/set_logger_level
/turtle1/set_pen
/turtle1/teleport_absolute
/turtle1/teleport_relative
/turtlesim/get_loggers
/turtlesim/set_logger_level
```

$ rosnode info /turtlesim

```
ros@ROS-VirtualBox:~$ rosnode info /turtlesim
--------------------------------------------------------------------------------
Node [/turtlesim]
Publications:
 * /rosout [rosgraph_msgs/Log]
 * /turtle1/color_sensor [turtlesim/Color]
 * /turtle1/pose [turtlesim/Pose]

Subscriptions:
 * /turtle1/cmd_vel [geometry_msgs/Twist]

Services:
 * /clear
 * /kill
 * /reset
 * /spawn
 * /turtle1/set_pen
 * /turtle1/teleport_absolute
 * /turtle1/teleport_relative
 * /turtlesim/get_loggers
 * /turtlesim/set_logger_level


contacting node http://ROS-VirtualBox:35153/ ...
Pid: 4355
Connections:
 * topic: /rosout
    * to: /rosout
    * direction: outbound (46185 - 127.0.0.1:56526) [26]
    * transport: TCPROS
 * topic: /turtle1/cmd_vel
    * to: /teleop_turtle (http://ROS-VirtualBox:41265/)
    * direction: inbound (48046 - ROS-VirtualBox:47331) [28]
    * transport: TCPROS
```

1.1 Turtlesim Examples

1.1.5 turtlesim服务

- rosservice call 语法：

```
ros@ROS-VirtualBox:~$ rosservice call --help
Usage: rosservice call /service [args...]

Options:
  -h, --help  show this help message and exit
  -v          print verbose output
  --wait      wait for service to be advertised
```

- 查看服务类型(消息类型)：

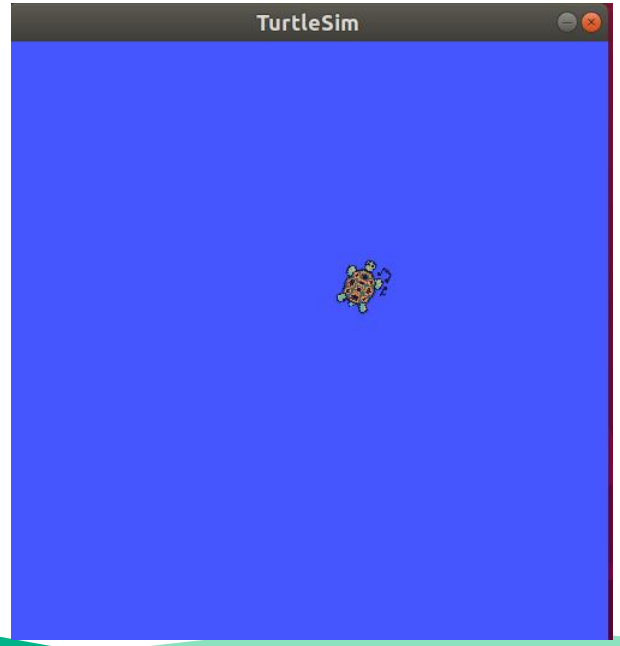
$ rosservice type /clear

```
ros@ROS-VirtualBox:~$ rosservice type /clear
std_srvs/Empty
ros@ROS-VirtualBox:~$ rossrv info std_srvs/Empty
---
```

- 调用服务：

$ rosservice call /clear

```
ros@ROS-VirtualBox:~$ rosservice call /clear
```



1.1 Turtlesim Examples

1.1.5 turtlesim服务

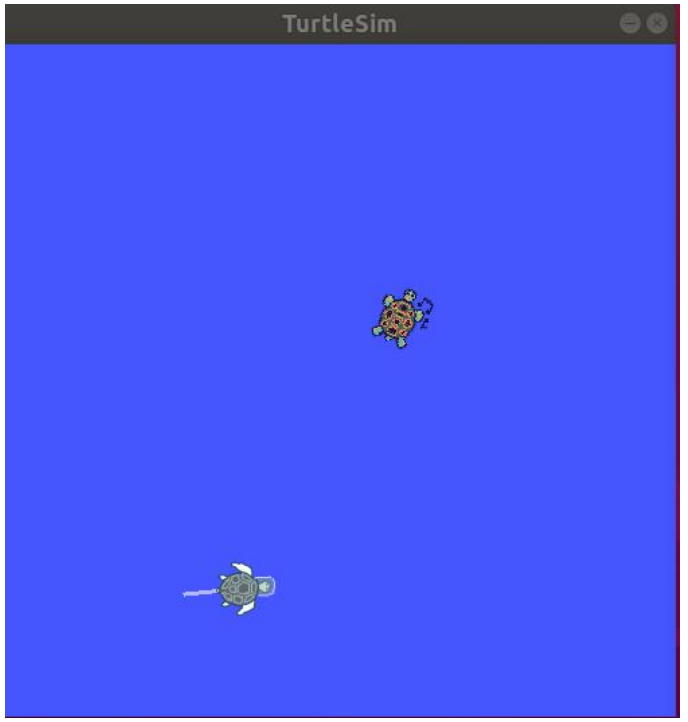
- 查看 /spawn 服务类型(消息类型)：

```
$ rosservice type /spawn | rossrv show
```

```
ros@ROS-VirtualBox:~$ rosservice type /s
pawn | rossrv show
float32 x
float32 y
float32 theta
string name
---
string name
```

- 调用/spawn服务：

```
$ rosservice call /spawn 3 2 0.1 "turtle2"
```



- 向话题/turtle2/cmd_vel发布信息，控制乌龟2运动：

```
$ rostopic pub /turtle2/cmd_vel geometry_msgs/Twist [1,0,0] [0,0,0]
```

1.1 Turtlesim Examples

1.1.6 turtlesim参数

- 查看可用参数：

```
$ rosparam list
```

```
ros@ROS-VirtualBox:~$ rosparam
 list
/rosdistro
/roslaunch/uris/host_ros_virtu
albox__46255
/rosversion
/run_id
/turtlesim/background_b
/turtlesim/background_g
/turtlesim/background_r
```

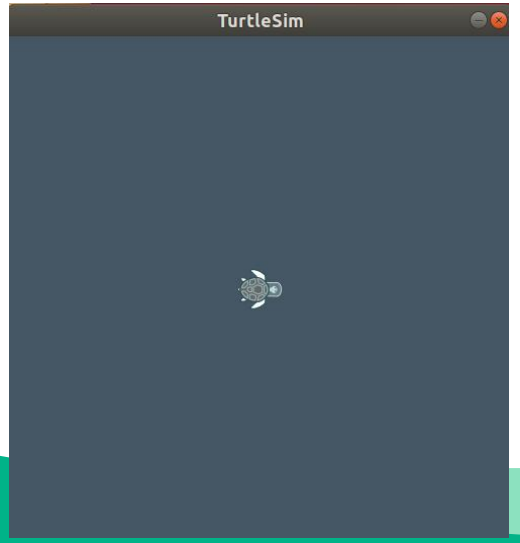
- 获取参数值：

```
$ rosparam get /turtlesim/background_b
```

```
ros@ROS-VirtualBox:~$ rosparam
 get /turtlesim/background_b
255
```

- 设置参数值：

```
$ rosparam set /turtlesim/background_b 100
```



1.1 Turtlesim Examples

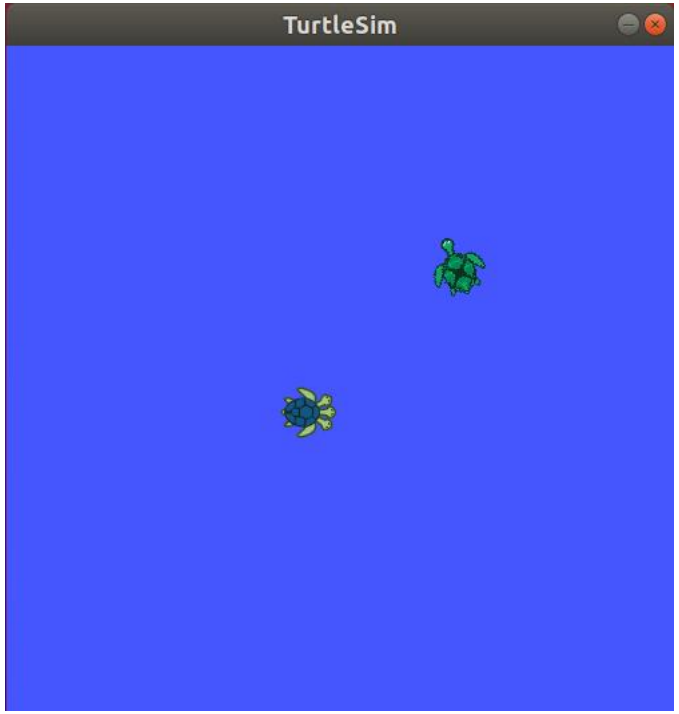
1.1.7 turtlesim mimic节点

- mimic节点的作用是让一个海龟复制另一个海龟的动作

- 启动另一只海龟：

```
$ rosservice call /spawn 5 5 0 "turtle_mimic"
```

```
ros@ROS-VirtualBox:~$ rosservice call /spawn
5 5 0 'turtle_mimic'
name: "turtle_mimic"
```

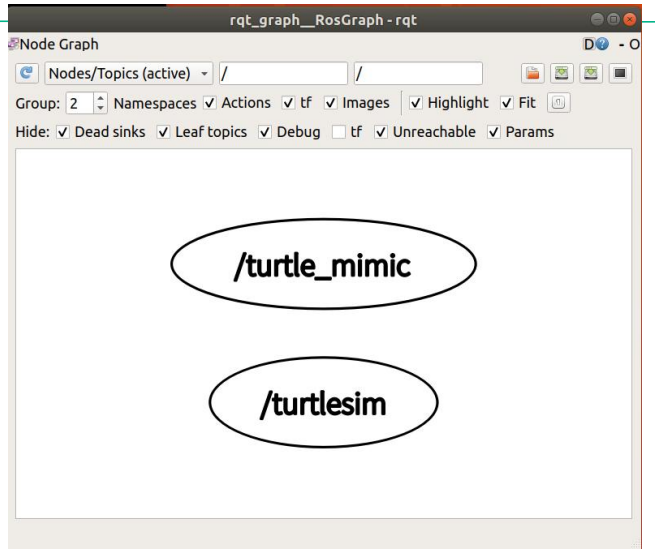


1.1 Turtlesim Examples

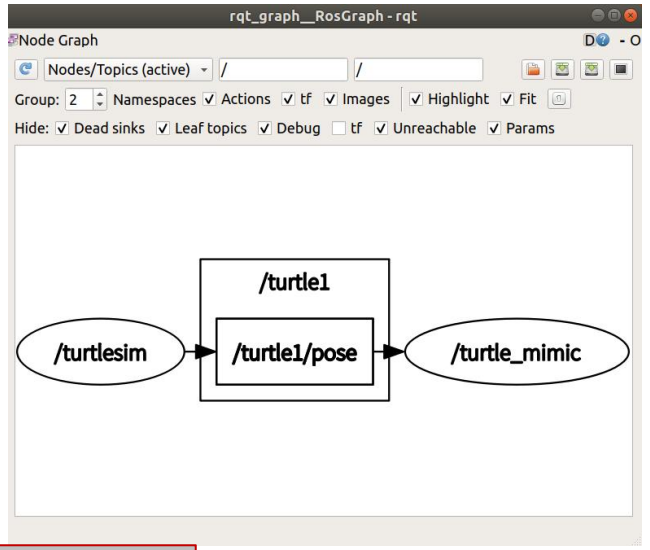
1.1.7 turtlesim mimic节点

- 启动mimic节点：

```
$ rosrun turtlesim mimic
```



- 为了实现动作复制，需要让 mimic 的输入为 turtle1 的 pose，输出为 turtle_mimic 的 cmd_vel，即将 mimic 原本的输入输出 topic 做如下修改：
  - /input/pose ----> /turtle1/pose
  - /output/cmd_vel ----> /turtle2/cmd_vel



- 关掉 mimic，再用如下方式重新启动 mimic：

```
$ rosrun turtlesim mimic input:=turtle1 output:=turtle_mimic
```

后边的参数表示将原程序中的两个参数 input, output，替换为 turtle1, turtle_mimic

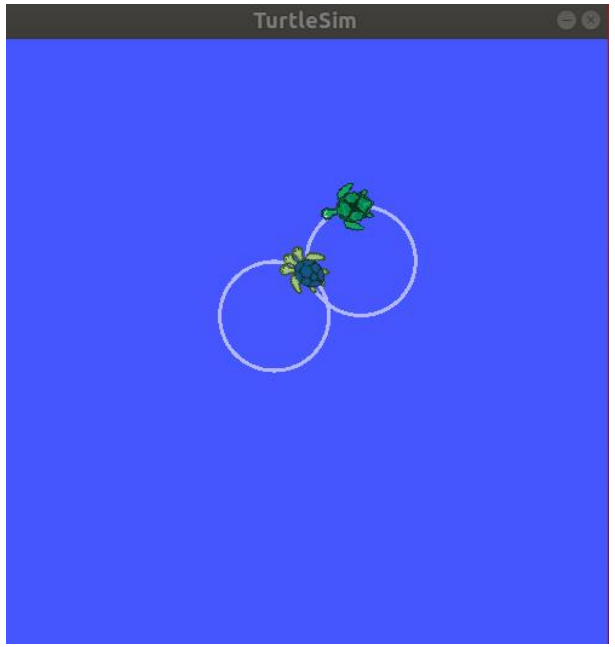
1.1 Turtlesim Examples

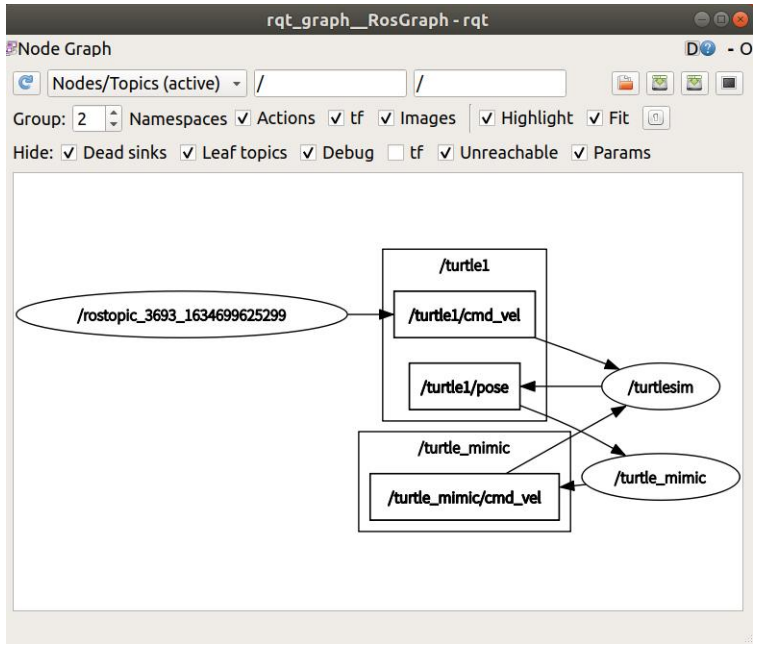
1.1.7 turtlesim mimic节点

- 手动向话题/turtle1/cmd_vel上发布消息：

```
$ rostopic pub /turtle1/cmd_vel geometry_msgs/Twist -r 1  [1,0,0] [0,0,1]
```

1秒/次速率循环发布





作业1：调用draw_square节点，使turtle1画矩形，生成海龟turtle_mimic，使turtle_mimic模仿turtle1的轨迹

参考资料：https://www.jianshu.com/p/5b8893479874

Summary

1. Turtlesim

a) turtlesim introduction, start ros, start turtlesim

b) turtlesim_node(rosnode rospotic) 

c) turtle_teleop_key(rqt, rosmsg, rossrv,rosparam) 

d) mimic(rosservice)

Homework



作业：学习《C++从入门到精通-作业版本》第8章

Q&A

今日事今日毕

Question?